Econ 104 Project

Mason Lee

2025-03-06

```
library(tidyverse)
library(ggplot2)
library(MASS)
library(dplyr)
library(corrplot)
library(forecast)
library(dynlm)
library(zoo)
library(vars)
library(tseries)
library(ARDL)
library(Metrics)
library(gridExtra)
```

Section 1

Data

```
data <- read_csv("data.csv")
```

```
## Rows: 1833 Columns: 10
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl  (9): SP500, Dividend, Earnings, Consumer Price Index, Long Interest Rat...
## date (1): Date
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
sandp <- data[c(1, 2, 6, 8, 9)]

sandp <- rename(sandp, "Interest" = "Long Interest Rate", "Div" = "Real Dividend", "Earn" = "Real Earni

sandp$SP500 <- as.numeric(sandp$SP500)
sandp$Earn <- as.numeric(sandp$Earn)
sandp$Interest <- as.numeric(sandp$Interest)
sandp$Div <- as.numeric(sandp$Div)

sandp <- sandp[-c(1830:1833), ]
```

Variables

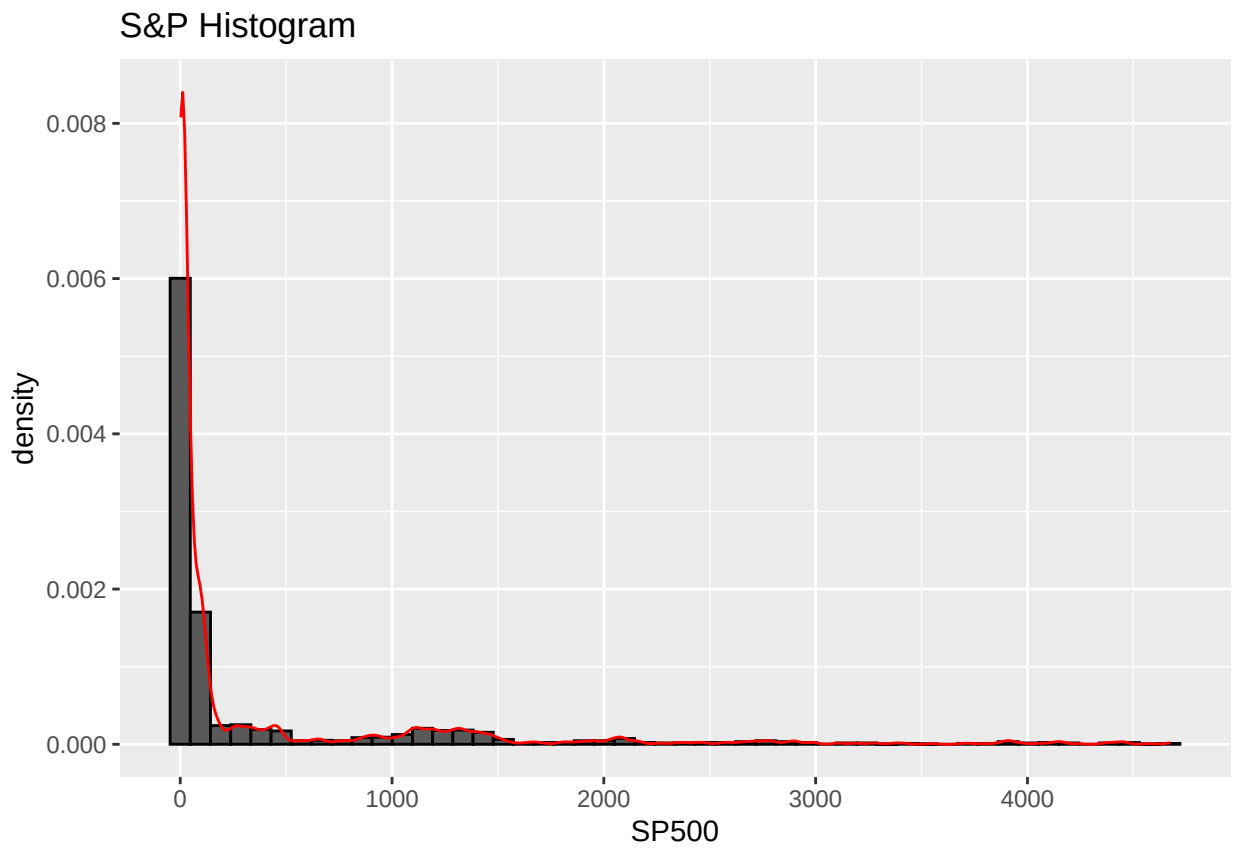
The dataset I will be using for this project is a monthly tracking of the S&P 500 stock price dating back to 1871. The dataset includes the price of the stock on the first of each month, but also includes the inflation rate for said month, the dividend payed by the stock, and the Real Earnings. These four numerical, continuous variables will be the center of the project. I will be using the S&P value and the Real Earnings as my y1t and y2t, and the Dividend payments and Long Interest Rate as the predictors. Because S&P and Earnings share dynamics, our VAR prediction should work well.

Question 1: Analysis of Variables

Histograms

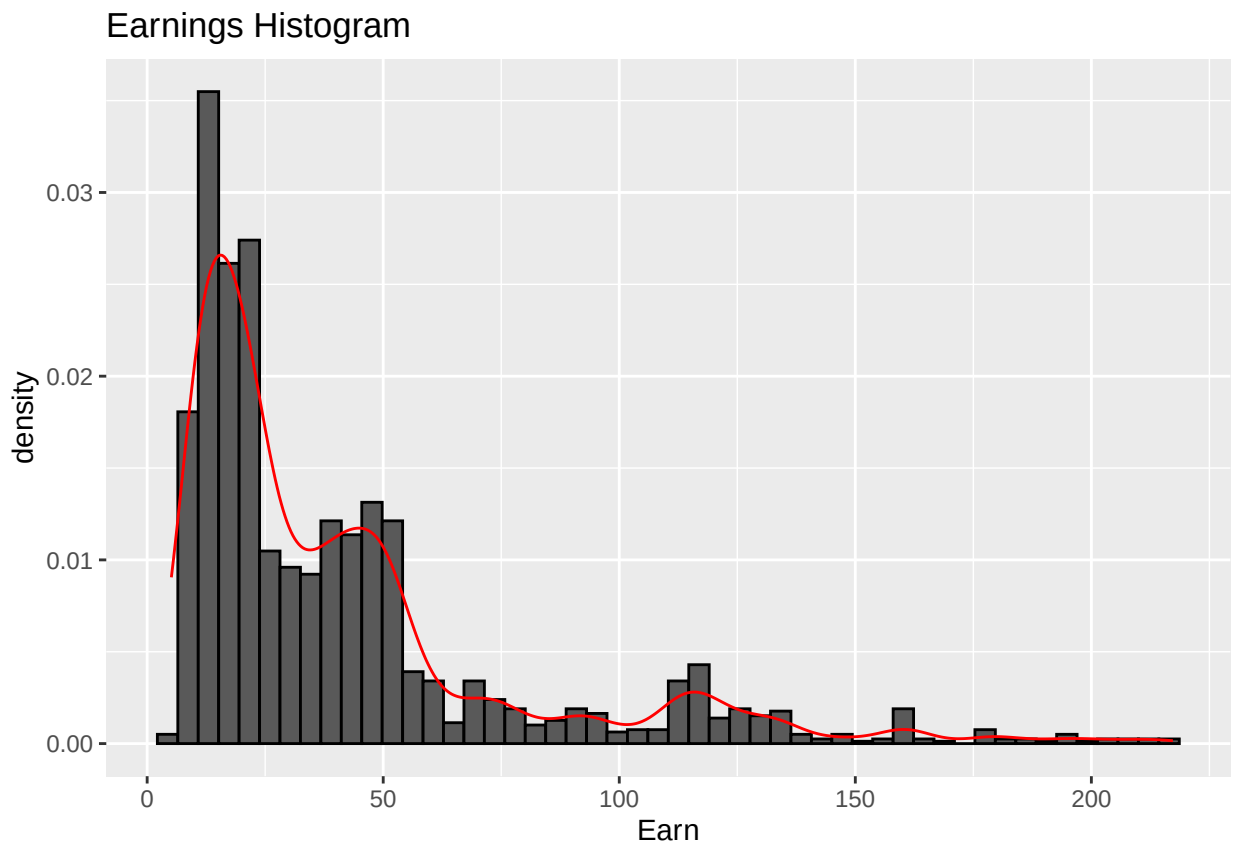
```
ggplot(data = sandp, aes(x = SP500)) +
  geom_histogram(aes(y = ..density..), bins = 50, color = "black") +
  geom_density(col = "red") +
  ggtitle("S&P Histogram")
```

```
## Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.
## i Please use `after_stat(density)` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



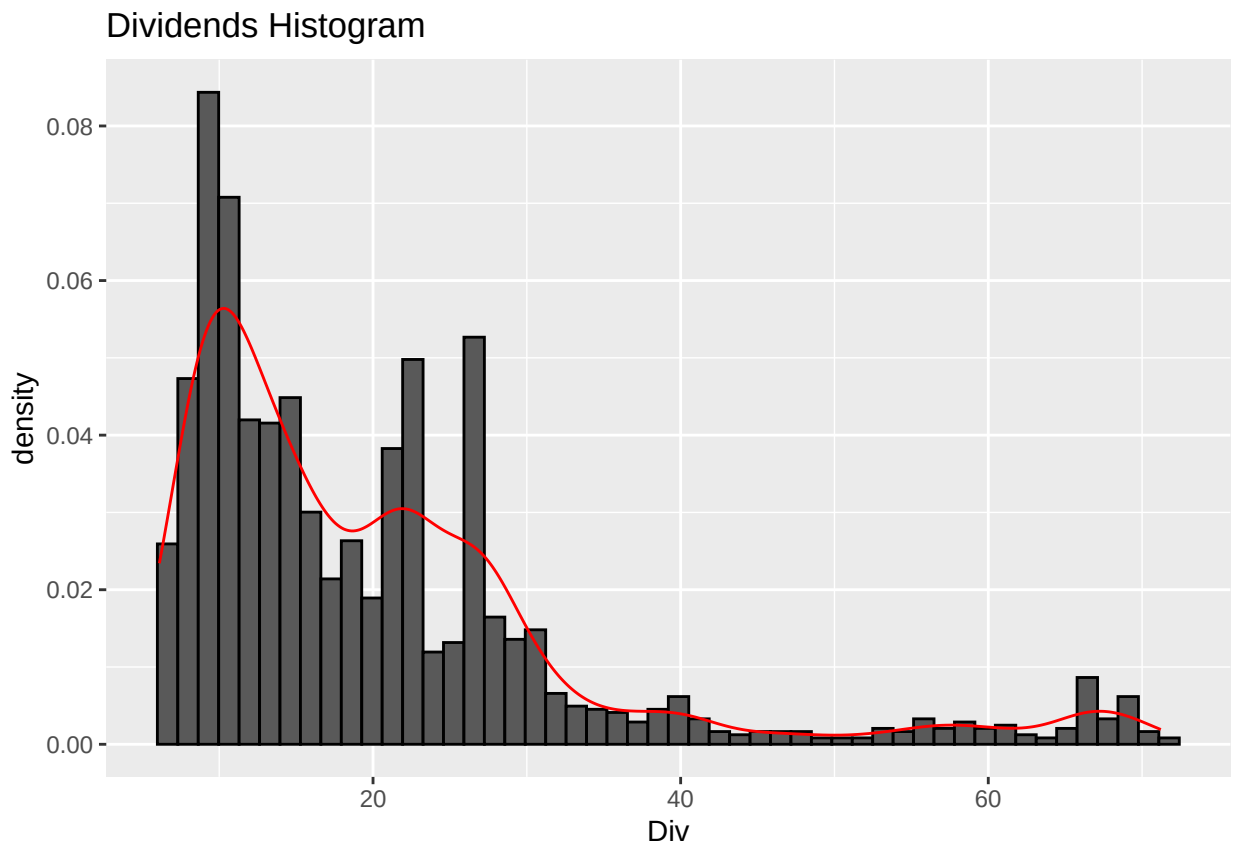
The S&P index is a continually rising figure, so the boxplot is heavily skewed to the right. The index has been around since 1871, and didn't break 100 points until 1979. Now it is at 3000. This exponential growth would be best represented by an exponential curve, which is what the red line most looks like (just reversed).

```
ggplot(data = sandp, aes(x = Earn)) +
  geom_histogram(aes(y = ..density..), bins = 50, color = "black") +
  geom_density(col = "red") +
  ggtitle("Earnings Histogram")
```



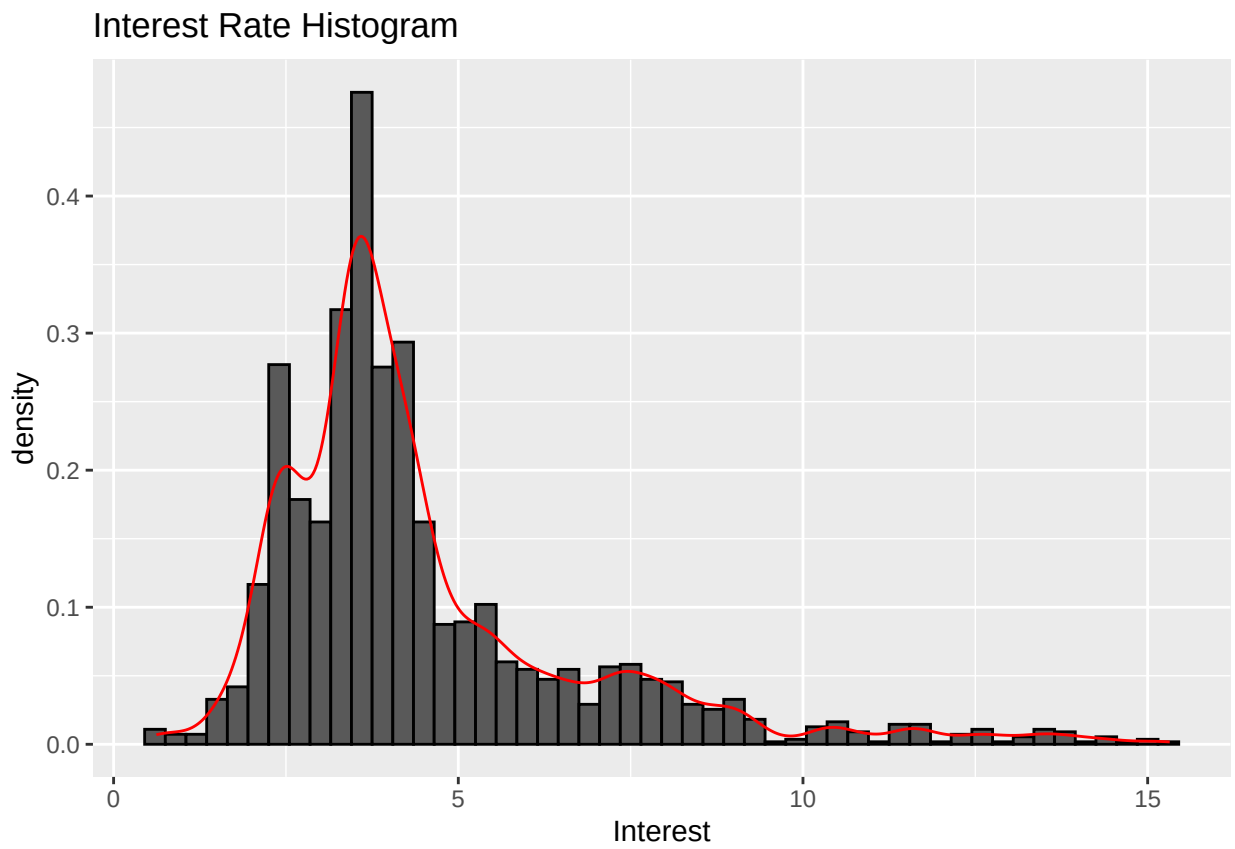
The Earnings data is also right skewed, but not as significantly as S&P. This density curve (red line) looks like a multi-tiered log-normal curve. It could also be described as multiple log-normal curves layered on top of each other. This would mean that each earning bracket experiences mirrored but higher earnings, and can definately be used in a model.

```
ggplot(data = sandp, aes(x = Div)) +
  geom_histogram(aes(y = ..density..), bins = 50, color = "black") +
  geom_density(col = "red") +
  ggtitle("Dividends Histogram")
```



The Dividends graph is very similar to the Earnings graph, which usually means a high correlation between the two. The density curve is doesn't quite reach the peaks (outliers or patterns may be present), but we see the same, right skewed pattern with mini peaks all the way down the line.

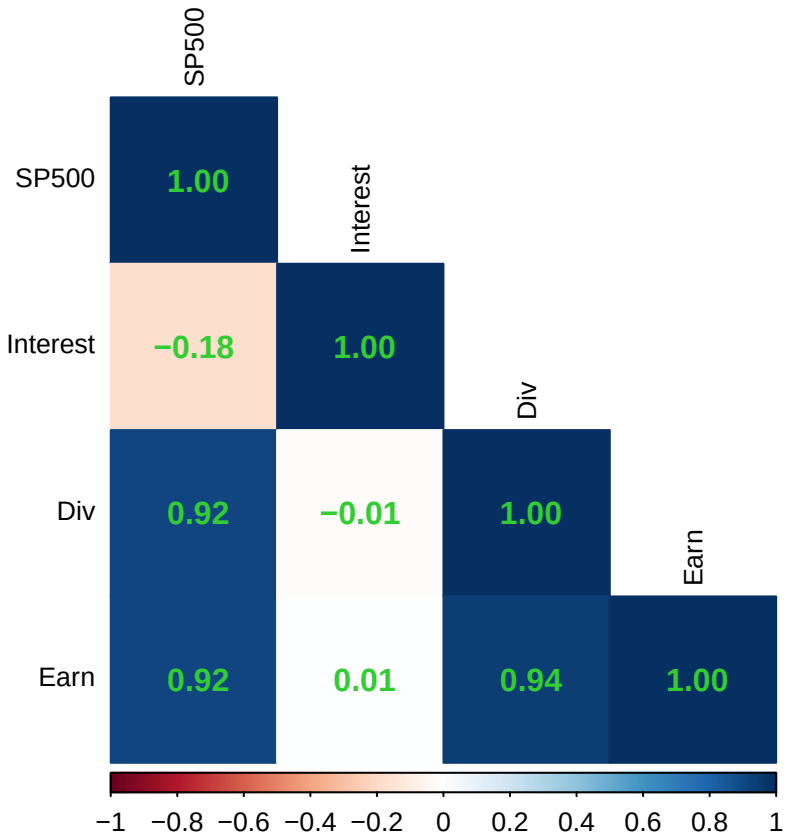
```
ggplot(data = sandp, aes(x = Interest)) +
  geom_histogram(aes(y = ..density..), bins = 50, color = "black") +
  geom_density(col = "red") +
  ggtitle("Interest Rate Histogram")
```



The Interest rate is supposed to be a number that doesn't change too quickly, so more central data is expected, and the histogram supports this. There is still a right tail, but the peak is much more normal this time, and almost all of the mini-peaks that we had seen down the right line in previous figures are now gone. For those reasons, Interest should prove to be a good predictor of SP500 and Earnings.

Correlation Plot

```
sandp2 <- sandp[, -1]

corr_matrix <- cor(sandp2)

corrplot(corr_matrix, method = "color", type = "lower", addCoef.col = "limegreen", tl.col = "black", tl
```

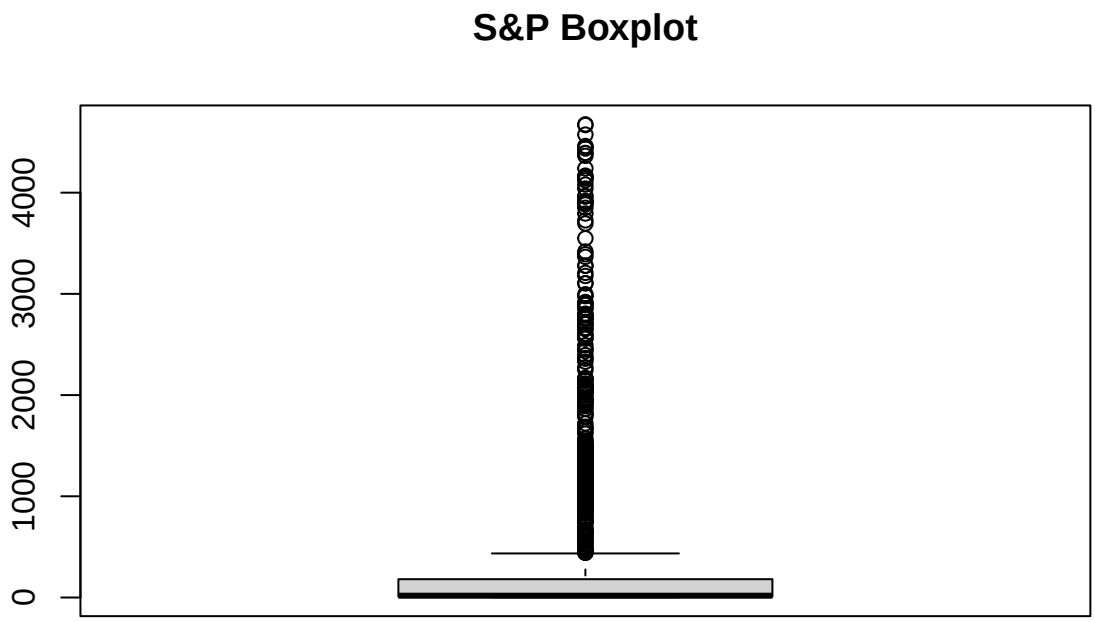


This Correlation Plot, generated with the corrplot library, provides a quick way to see how closely variables are correlated with each other. The darker the color, the more significant the correlation. I used a new dataset (sandp2) for this problem to eliminate the date variable from the chart.

Each variable is fairly correlated to each other. The least significant correlation is found between the real interest rate and each of the other three variables. It is good to see a high correlation between S&P and Earn, as these are our two y values for the VAR test. We also see a negative corelation between the Interest Rate and the S&P. This is logically correct because when the interest rate is higher, people can make more money leaving their savings in the bank compared to investing, and prices should go down as a result. Interest Rates and Dividend payments have no relationship with each other, but this is because a dividend payment only reflects the stock price and not the rest of the market (same logic for Earn and Interest). Overall, this correlation graph is very promising.

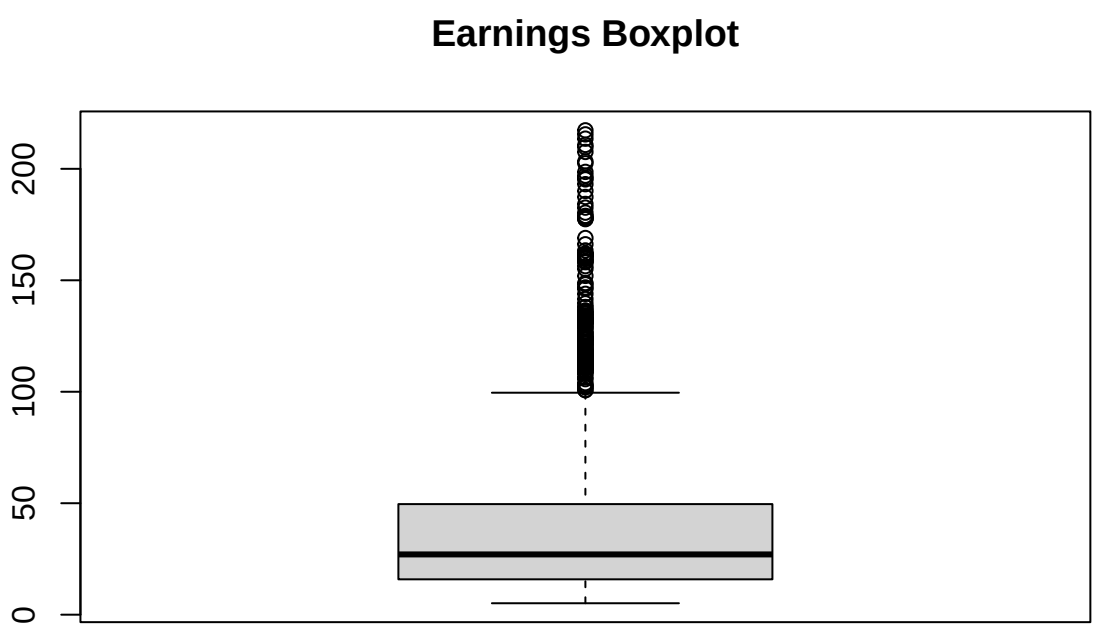
Boxplot

```
boxplot(sandp$SP500, main = "S&P Boxplot")
```



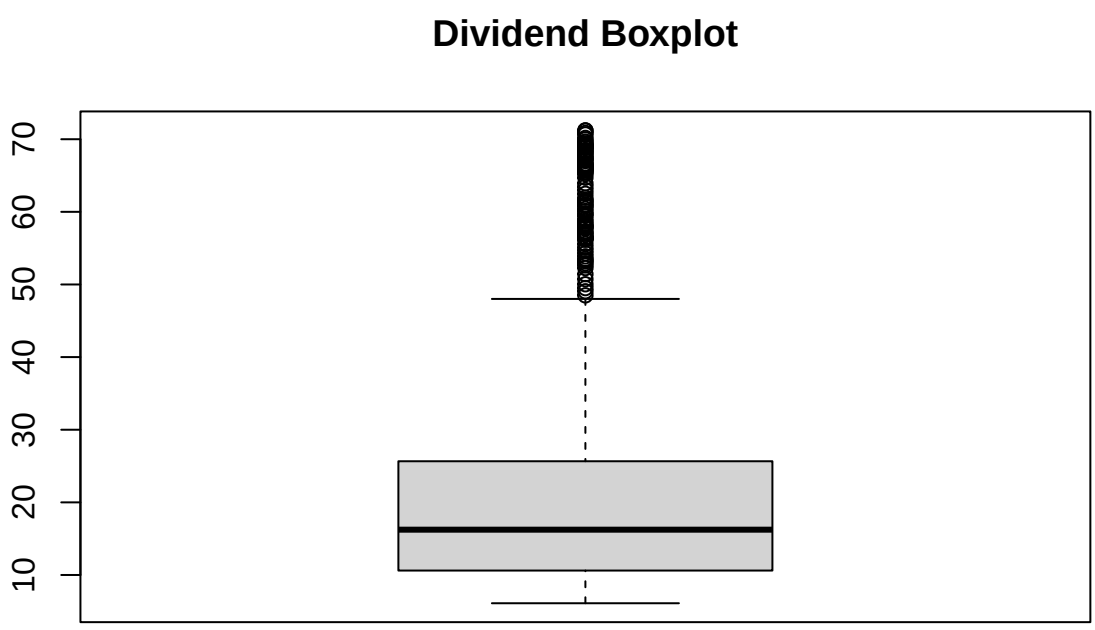
Due to the nature of the market (increasing exponentially), the median is very low compared to current stock prices. Compounding numbers monthly will always lead to a misleading median, so a boxplot is not a good representation for this variable

```
boxplot(sandp$Earn, main = "Earnings Boxplot")
```



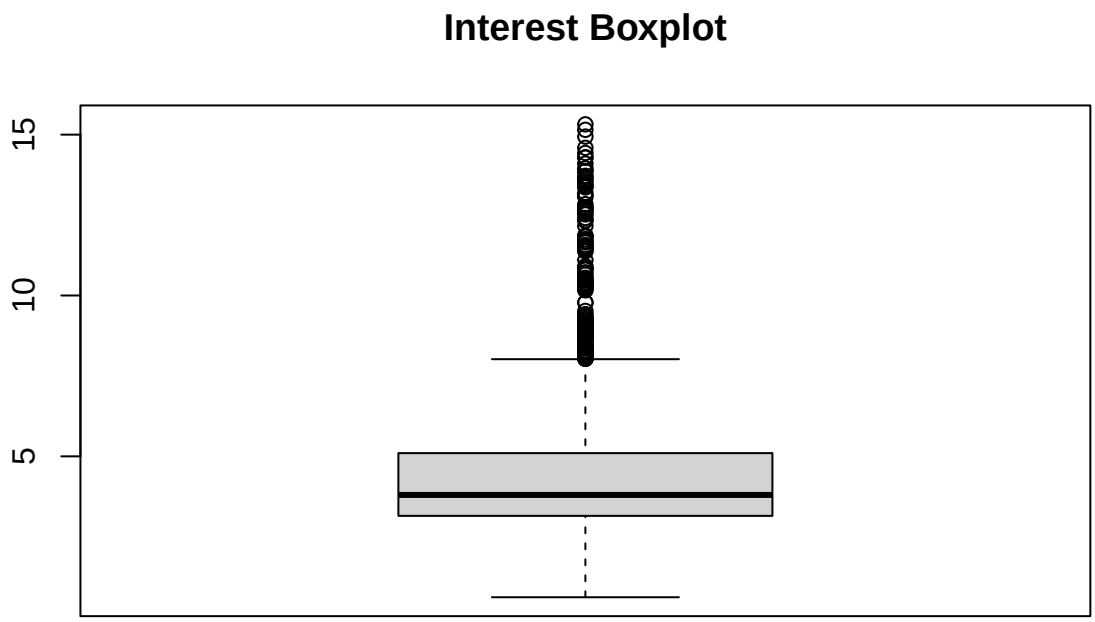
Similar to S&P, the Earnings is steadily increasing over time, but less significantly. There are many less outliers here than in the previous boxplot, and the quartiles have a little bit more meaning. The median is still misleading, but this boxplot does show that CPI grows much slower than the S&P.

```
boxplot(sandp$Div, main = "Dividend Boxplot")
```



This boxplot shows much more than the prior two plots because the dividends don't rely much on the time. The Dividend payment fluctuates more, and has a cyclical nature. The mediuan here is around 12 dollars, with anything over 50 being considered an outlier. Half of the 1833 dividend payments fall between 10 and 25 dollars.

```
boxplot(sandp$Interest, main = "Interest Boxplot")
```

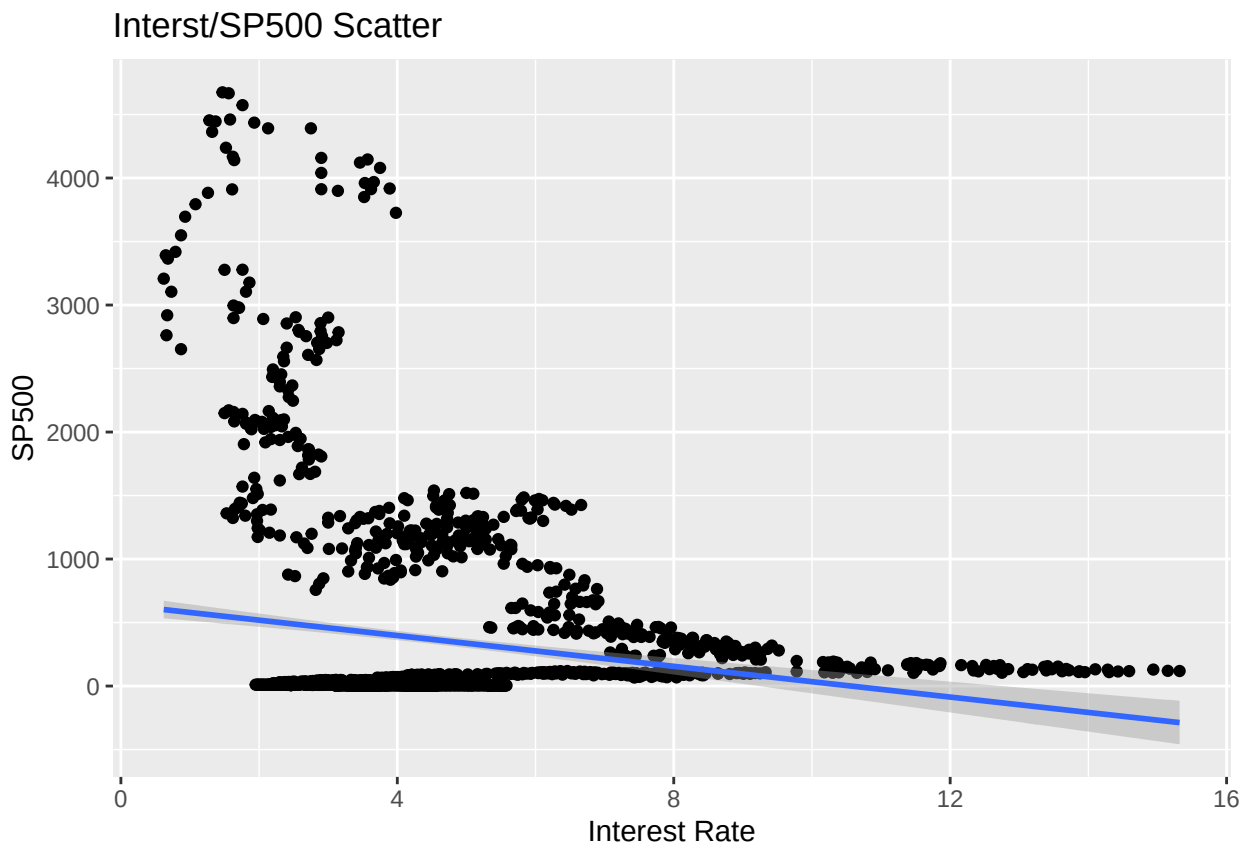


Interest and the S&P are mostly independent. One can be an indication of the other, but the only way one can influence the other is if the numbers affect someones investment decision. Interest rates tend to be constant around 3%, and that can be seen here. A few outliers (anything about 7.5%) push the median to 4%, but half of the data falls between 3 and 5% historically.

Scatter Plots

```
ggplot(data = sandp, aes(x = Interest, y = SP500)) +
  geom_point() +
  geom_smooth(method = "lm") +
  ggtitle("Interst/SP500 Scatter") +
  labs(x = "Interest Rate", y = "SP500")
```

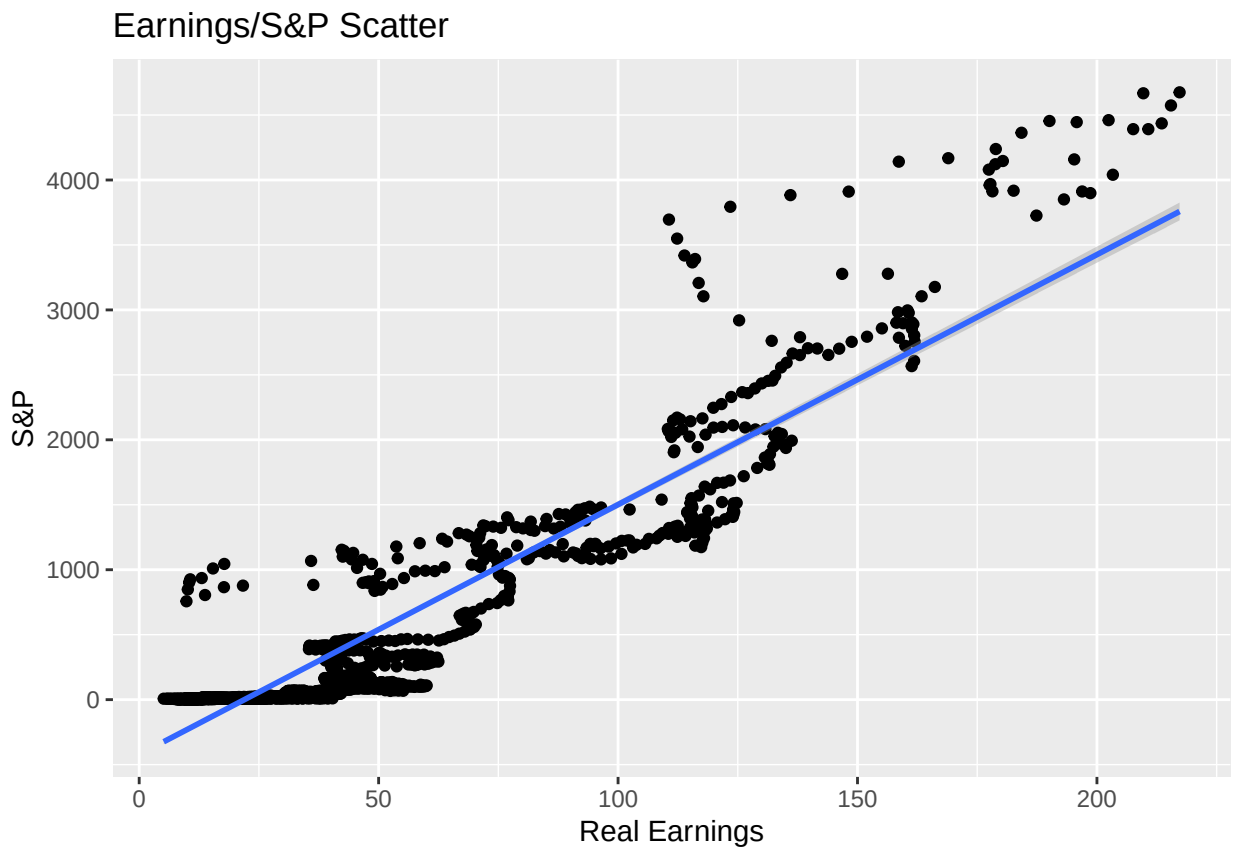
```
## `geom_smooth()` using formula = 'y ~ x'
```



As dictated by our correlation figure, a lower interest rate leads to a higher SP500 figure. The blue line of best fit reflects this negative relationship. The corelation is weak but the pattern is clear.

```
ggplot(data = sandp, aes(x = Earn, y = SP500)) +
  geom_point() +
  geom_smooth(method = "lm") +
  ggtitle("Earnings/S&P Scatter") +
  labs(x = "Real Earnings", y = "S&P")
```

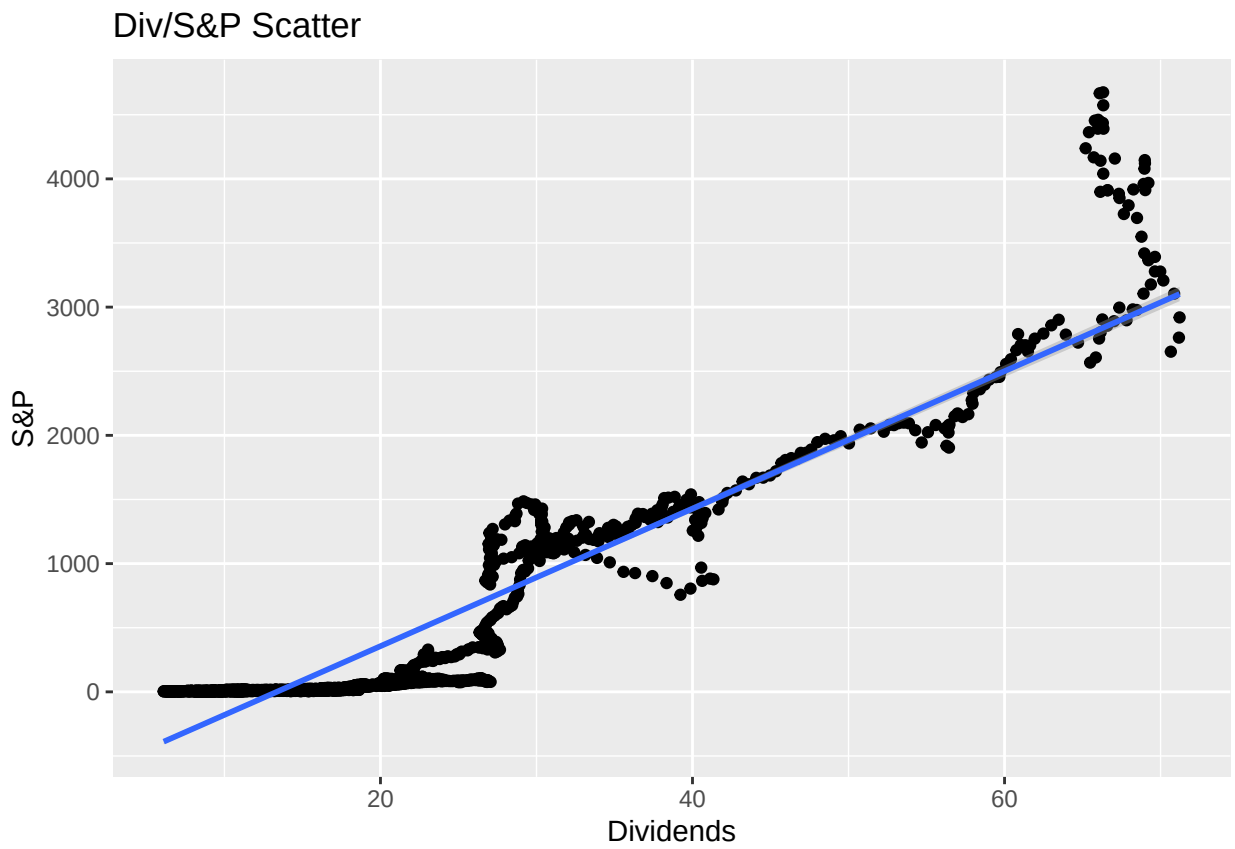
```
## `geom_smooth()` using formula = 'y ~ x'
```



Real Earnings and the S&P are highly correlated, and the 0.94 slope of the best fit line proves this, but many things other than the S&P monthly rates go into Earnings, as can be seen by the curvy nature of the data. Although trending upwards, there are clear deviations in this graph that will provide an interesting twist to our model.

```
ggplot(data = sandp, aes(x = Div, y = SP500)) +
  geom_point() +
  geom_smooth(method = "lm") +
  ggtitle("Div/S&P Scatter") +
  labs(x = "Dividends", y = "S&P")
```

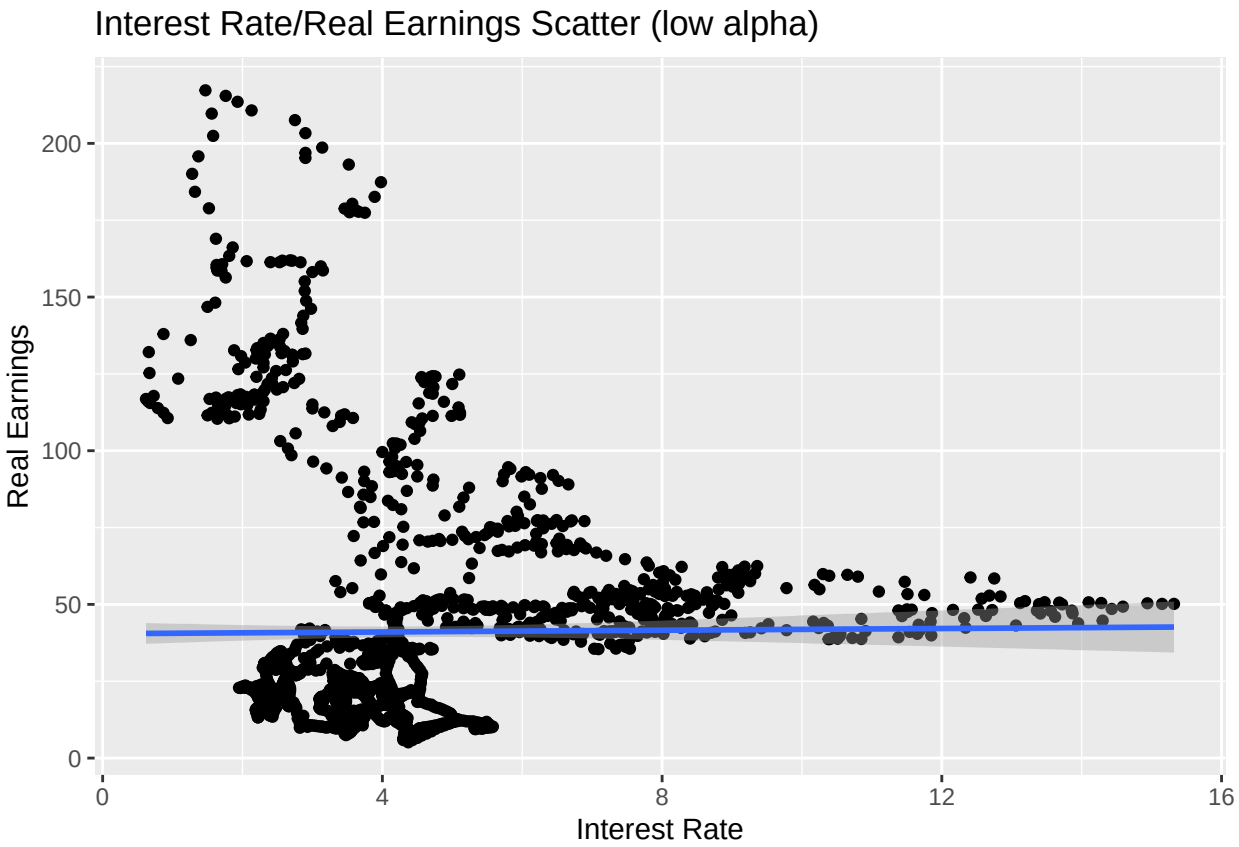
```
## `geom_smooth()` using formula = 'y ~ x'
```



This graph features a similar pattern to the one directly above, with a sharp rise towards the end. This is because it takes many years to build the curving figure seen snaking up the line of best fit, and the S&P just recently hit 3000+ points. Dividend payments are very constant at these newer levels despite the S&P fluctuating.

```
ggplot(data = sandp, aes(x = Interest, y = Earn)) +
  geom_point() +
  geom_smooth(method = "lm") +
  ggtitle("Interest Rate/Real Earnings Scatter (low alpha)") +
  labs(x = "Interest Rate", y = "Real Earnings")
```

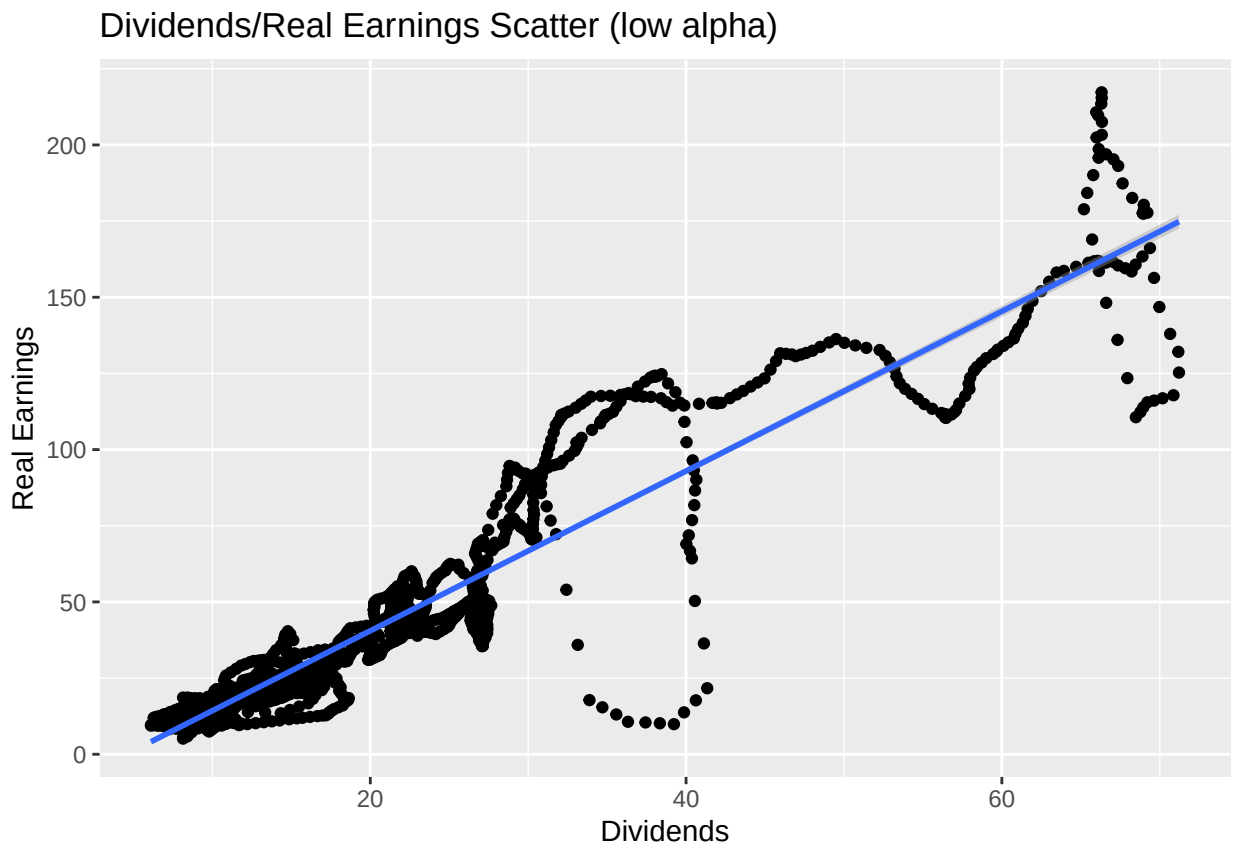
```
## `geom_smooth()` using formula = 'y ~ x'
```



There does not seem to be a real pattern between Earnings and the Interest Rate. It seems that earnings are constant at around 50 when the Interest rate is in outlier territory, but when it settles down Earnings seem to fluctuate. My prediction is that this is because of the random walk of stock prices when interest is standard, but more analysis is needed to prove this.

```
ggplot(data = sandp, aes(x = Div, y = Earn)) +
  geom_point() +
  geom_smooth(method = "lm") +
  ggtitle("Dividends/Real Earnings Scatter (low alpha)") +
  labs(x = "Dividends", y = "Real Earnings")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Dividends and Earnings have a very strong, positive correlation. They don't directly predict each other, but provide useful information. Because of a stocks random walk, we see the upwards snaking pattern, but with more clarity in the more recent, higher paying years.

5 Number Summary

The 5 Number Summaries tend to show similar information as the Boxplots. The middle three numbers (1st Quartile, Median, 3rd Quartile), are exactly the same. The first and last numbers are the variables minimum and maximum value.

```
fivenum(sandp$SP500)
```

```
## [1]    2.730    7.930   18.020  180.900 4674.773
```

```
fivenum(sandp$Earn)
```

```
## [1]   5.13  15.86  27.03  49.60 217.26
```

```
fivenum(sandp$Div)
```

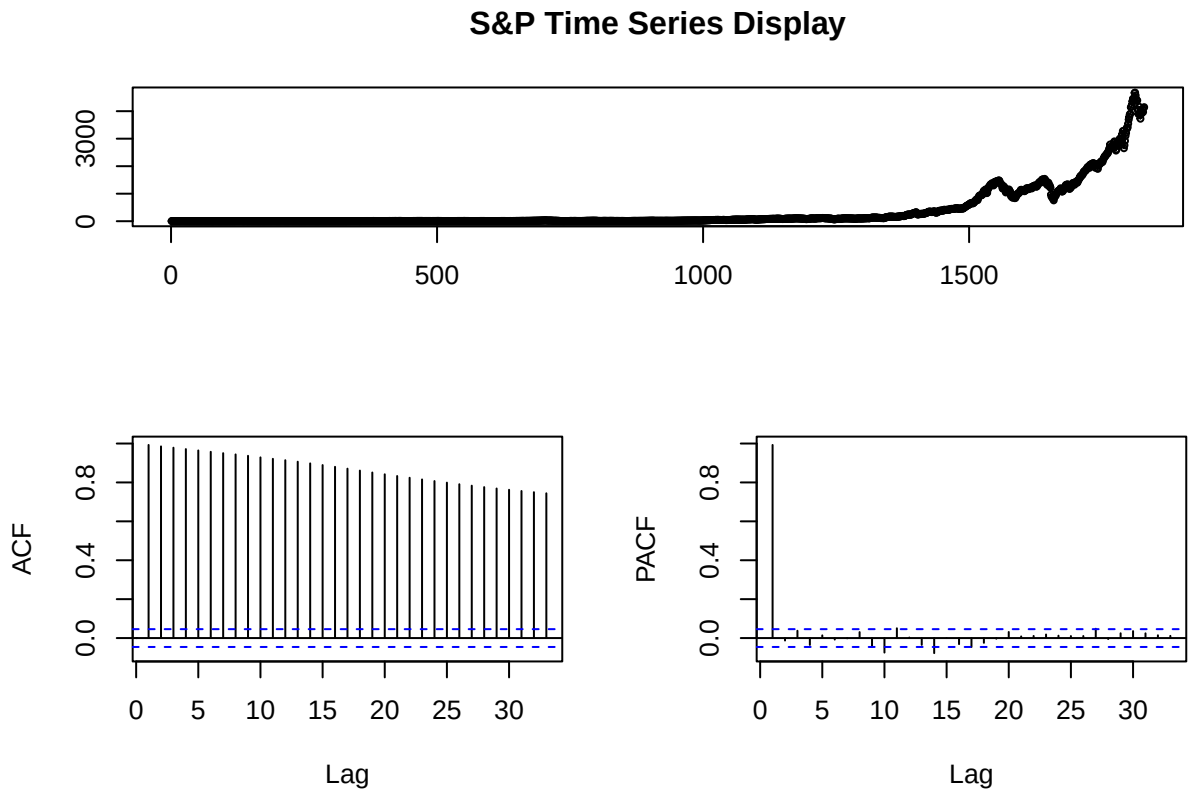
```
## [1]  6.11 10.61 16.24 25.66 71.22
```

```r
fivenum(sandp$Interest)
```

```
## [1]  0.62  3.15  3.80  5.10 15.32
```

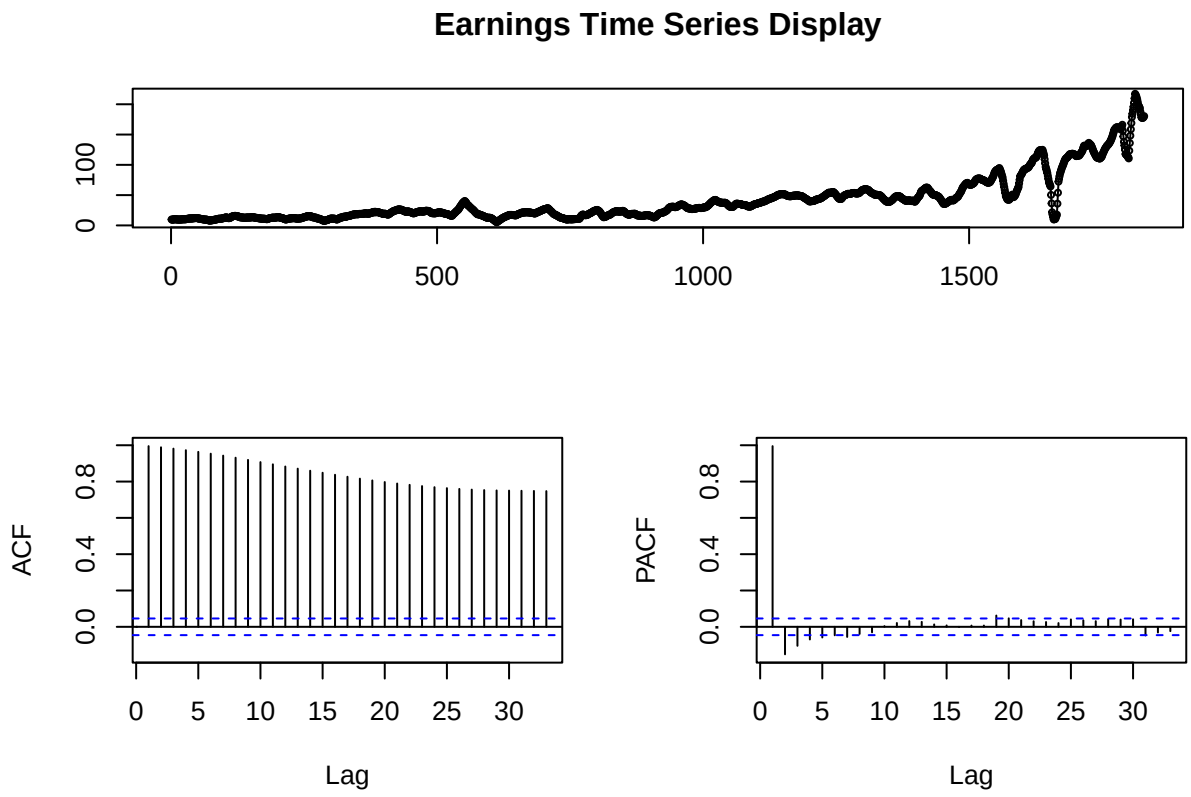
Question 2: Time Series Displays

```r
sp_ts <- ts(sandp$SP500)

tsdisplay(sp_ts, main = "S&P Time Series Display")
```



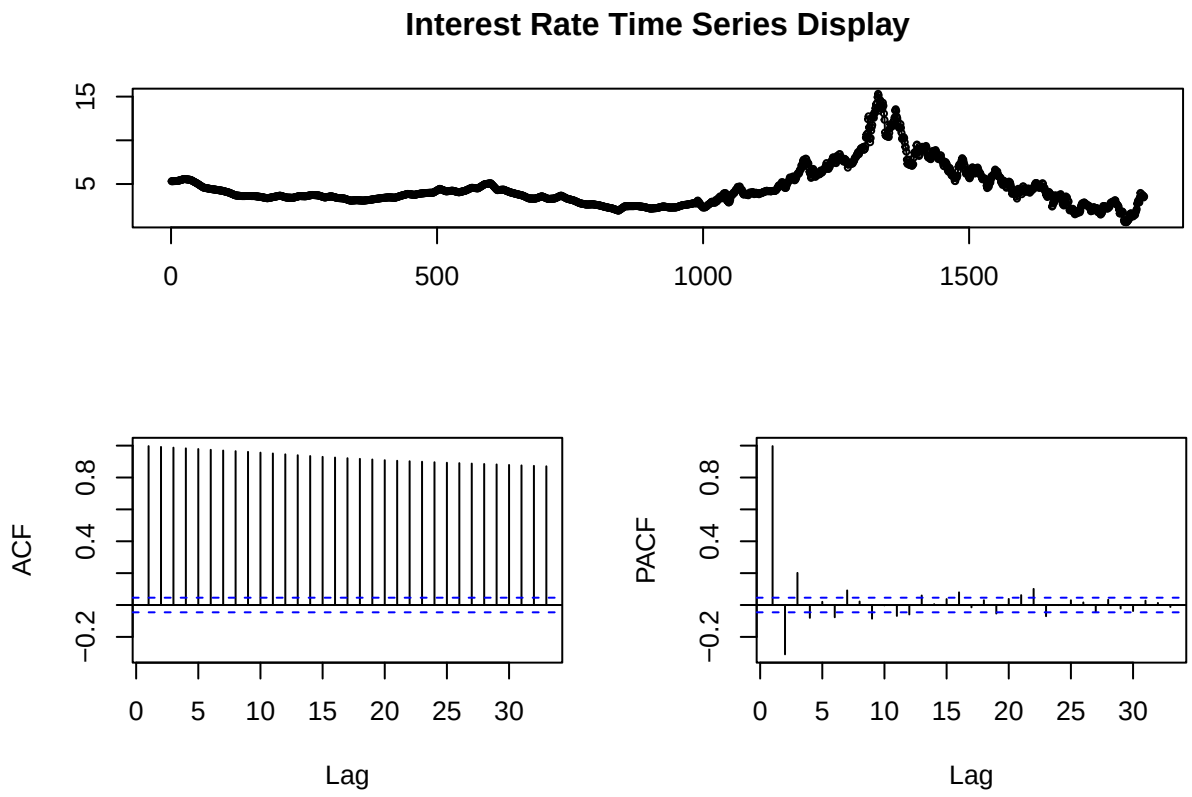
The top graph shows the path that S&P takes, which is very exponential as predicted. There are peaks and valleys, but the overall trend is rapid and upwards. The ACF graph shows a very slow and steady decline, which usually points towards an AR model. The PACF shows a sharp decline, with only the first vertical line crossing the dotted thresholds, so an AR(1) model would be appropriate here.

```r
earn_ts <- ts(sandp$Earn)

tsdisplay(earn_ts, main = "Earnings Time Series Display")
```



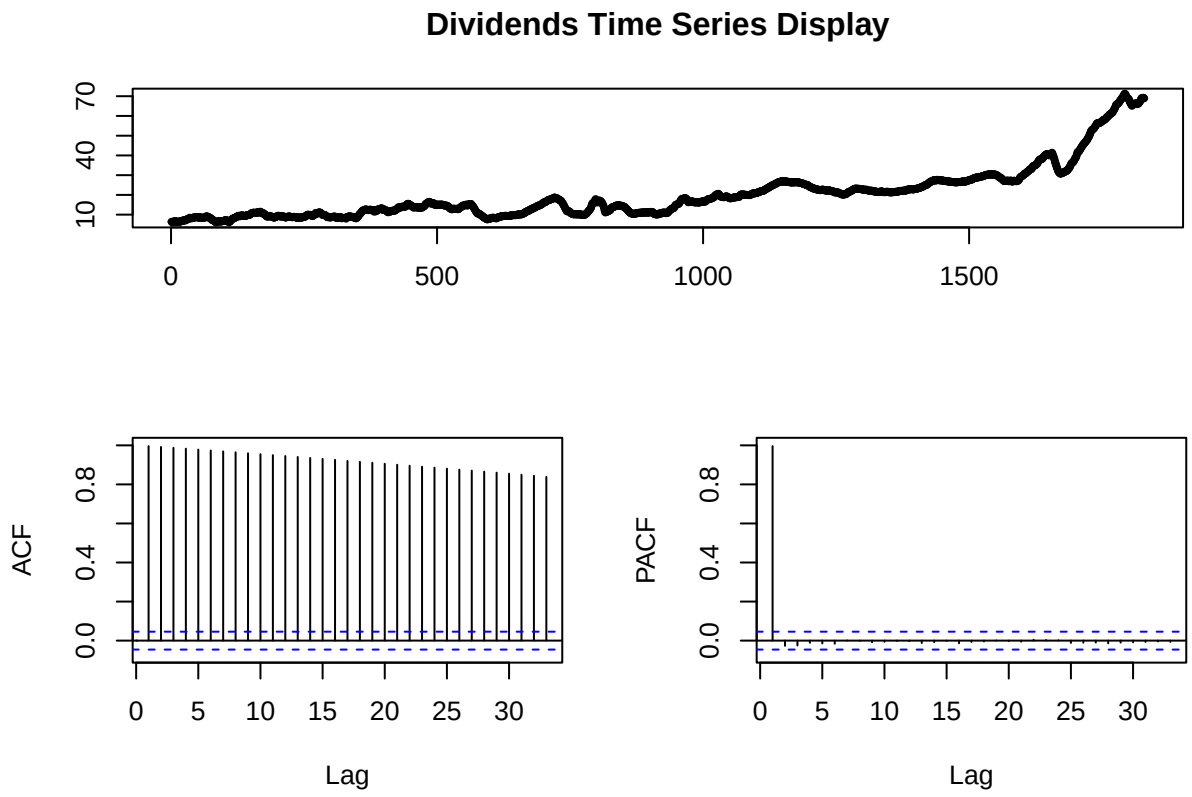
The Earnings variable also features an upward trend, but it is much rockier than SP500. It is still not a random walk, but the exponential growth is a little less apparent, especially if you zoom in on a specific time window instead of looking at the trend as a whole. The ACF graph is also slowly declining, but the PACF has more info than the previous TS Display. Here, the sharp decline is followed by a gradual, negative slope into the threshold. The first 3 lags are significant, but the sharp dropoff happens after lag 1, so further inspection will be needed to find an appropriate model.

```
interest_ts <- ts(sandp$Interest)

tsdisplay(interest_ts, main = "Interest Rate Time Series Display")
```



The Interest TS graph shows a seemingly random pattern with a mostly constant beginning and a very spiky, downward sloping end. The ACF trends gradually downwards, but the PACF has a clearer dropoff in this case. There are 3 significant lags before the lines start falling into the threshold, so an AR(3) model would be appropriate.

```
div_ts <- ts(sandp$Div)

tsdisplay(div_ts, main = "Dividends Time Series Display")
```



This TS graph looks very similar to the Earnings trend, with a spiky exponential pattern appearing. Much like the previous three, the ACF plot is slowly declining. In this case, the PACF has a very sharp drop off after 1 lag, so an AR(1) would be best.

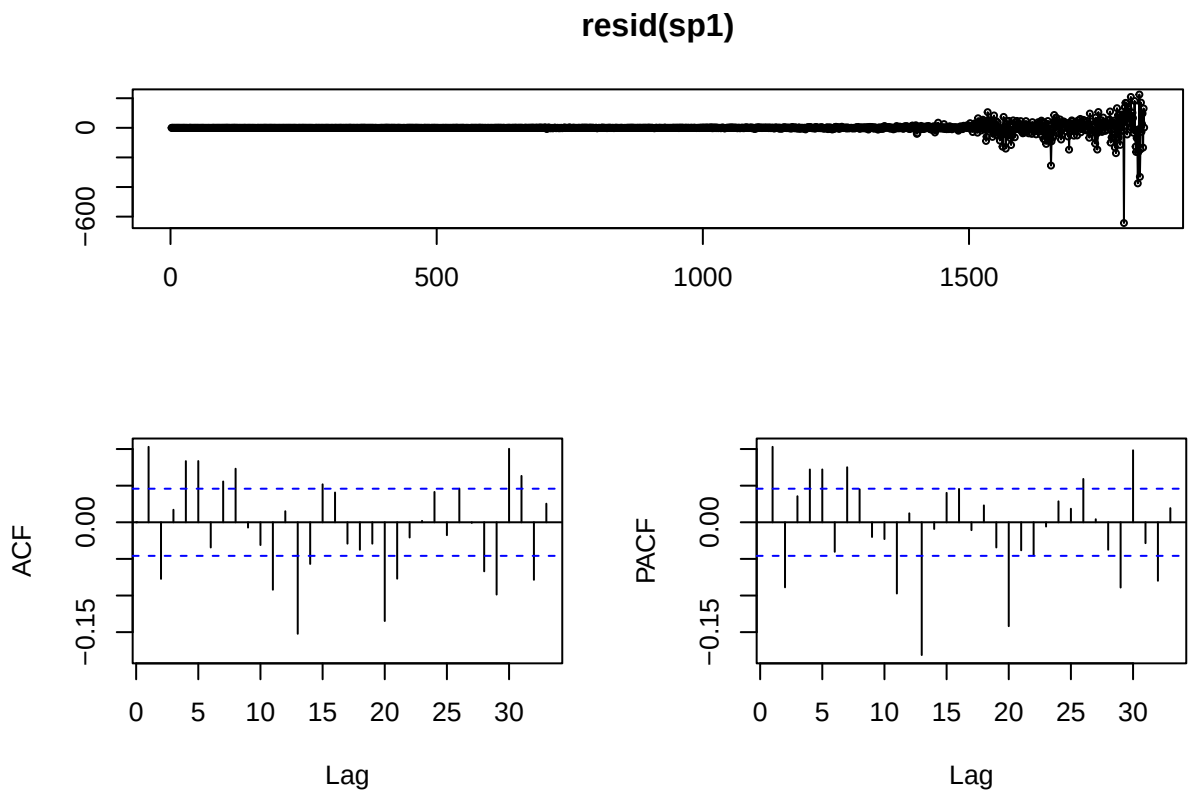
Question 3

Models

```
sp1 <- dynlm(sp_ts ~ L(sp_ts, 1))
sp2 <- dynlm(sp_ts ~ L(sp_ts, 1) + L(sp_ts, 2))

earn1 <- dynlm(earn_ts ~ L(earn_ts, 1) + L(earn_ts, 2))
earn2 <- dynlm(earn_ts ~ L(earn_ts, 1) + L(earn_ts, 2) + L(earn_ts, 3))

int1 <- dynlm(interest_ts ~ L(interest_ts, 1) + L(interest_ts, 2) + L(interest_ts, 3))
int2 <- dynlm(interest_ts ~ L(interest_ts, 1) + L(interest_ts, 2) + L(interest_ts, 3) + L(interest_ts, 4

div1 <- dynlm(div_ts ~ L(div_ts, 1))
div2 <- dynlm(div_ts ~ L(div_ts, 1) + L(div_ts, 2))
```

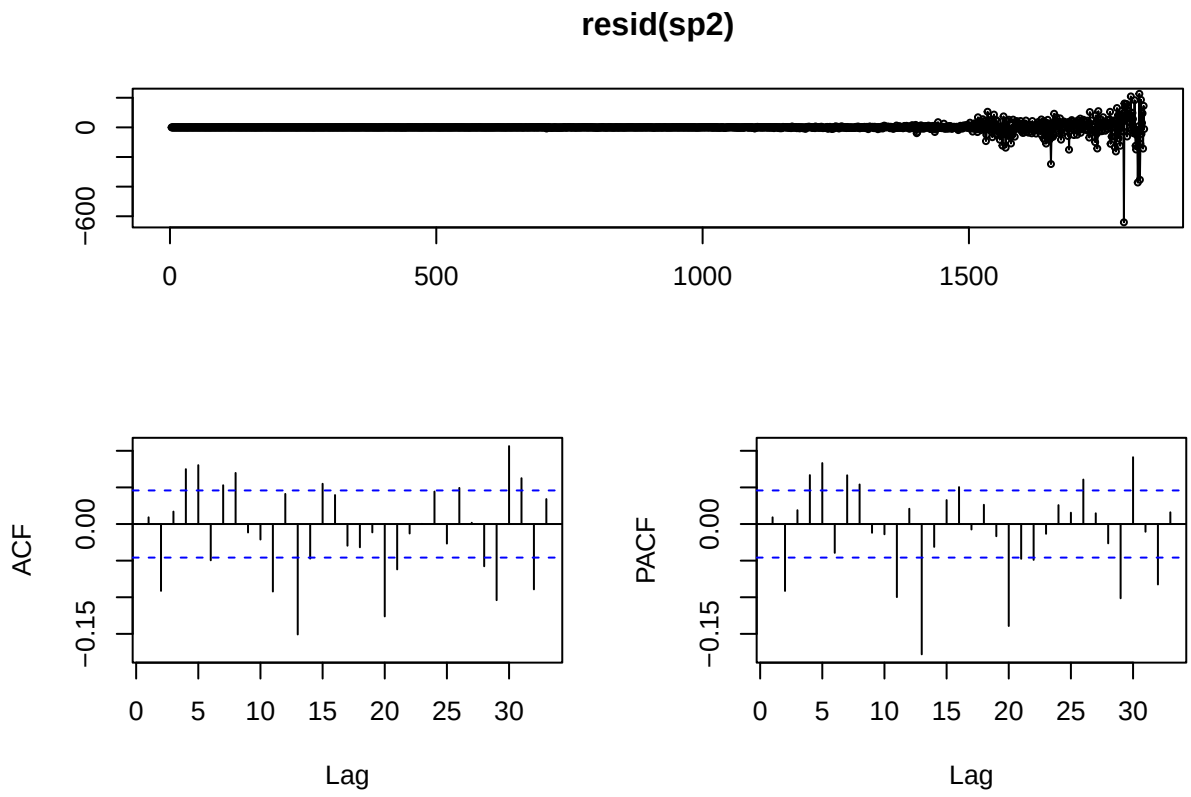
Residuals

```
tsdisplay(resid(sp1))
```



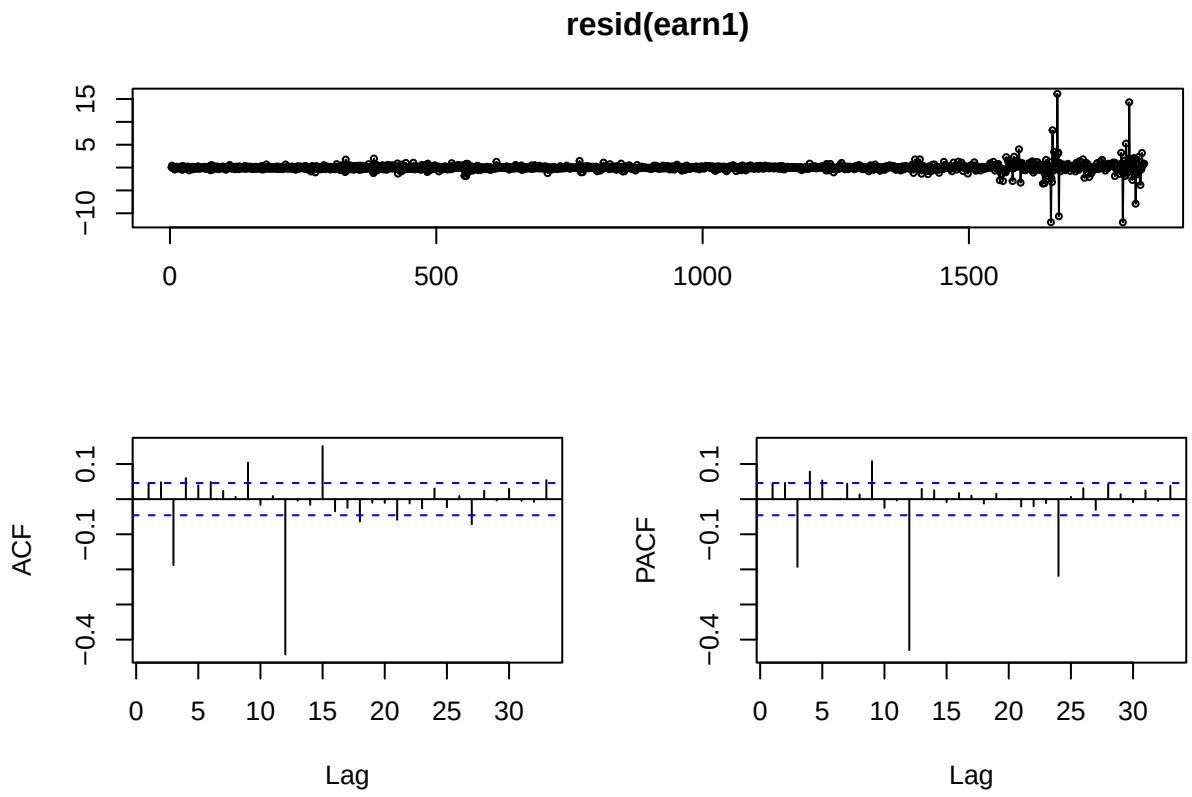
The oscilations in the ACF and PACF outputs suggest possible seasonal patterns, like stocks spiking around certain months. The PACF seems to be declining more than the ACF, so an MA may be appropriate.

```
tsdisplay(resid(sp2))
```



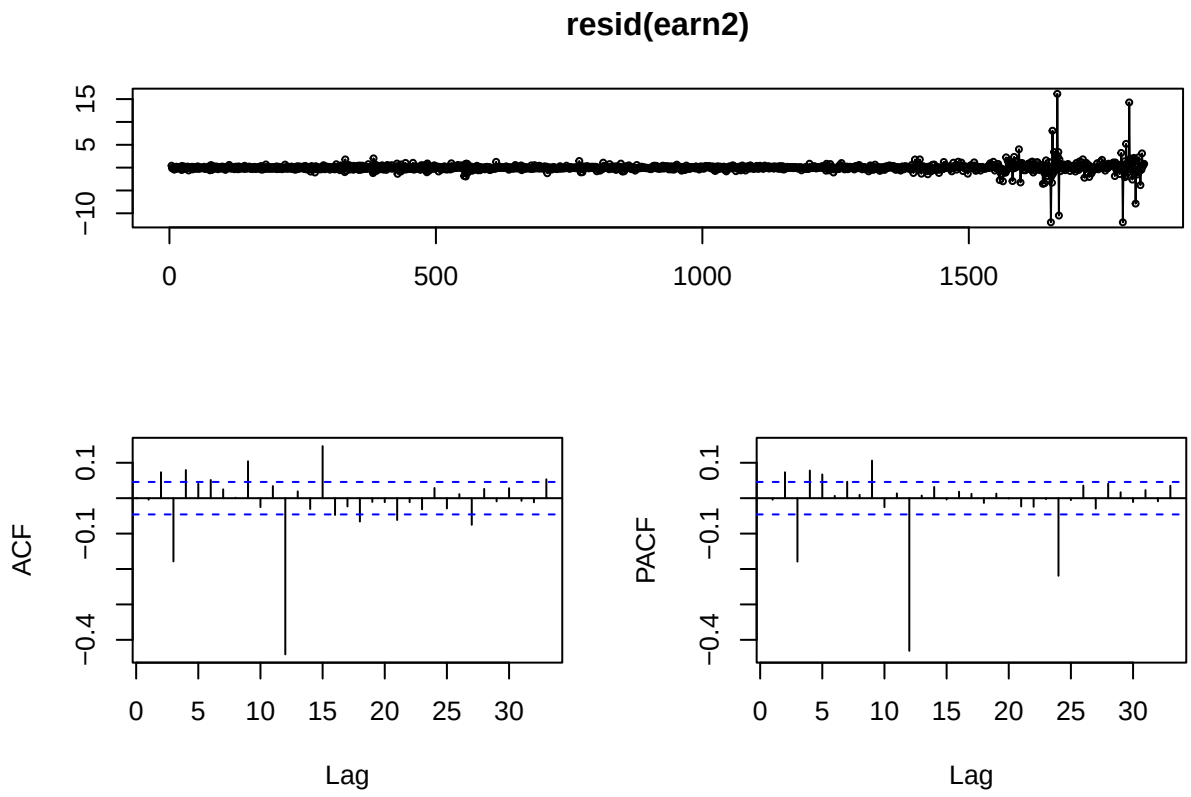
The only difference between this model and the AR(1) model above is that the PACF starts out insignificant, meaning the first model is probably more accurate, and adding in the extra lag hurts the efficacy.

```
tsdisplay(resid(earn1))
```



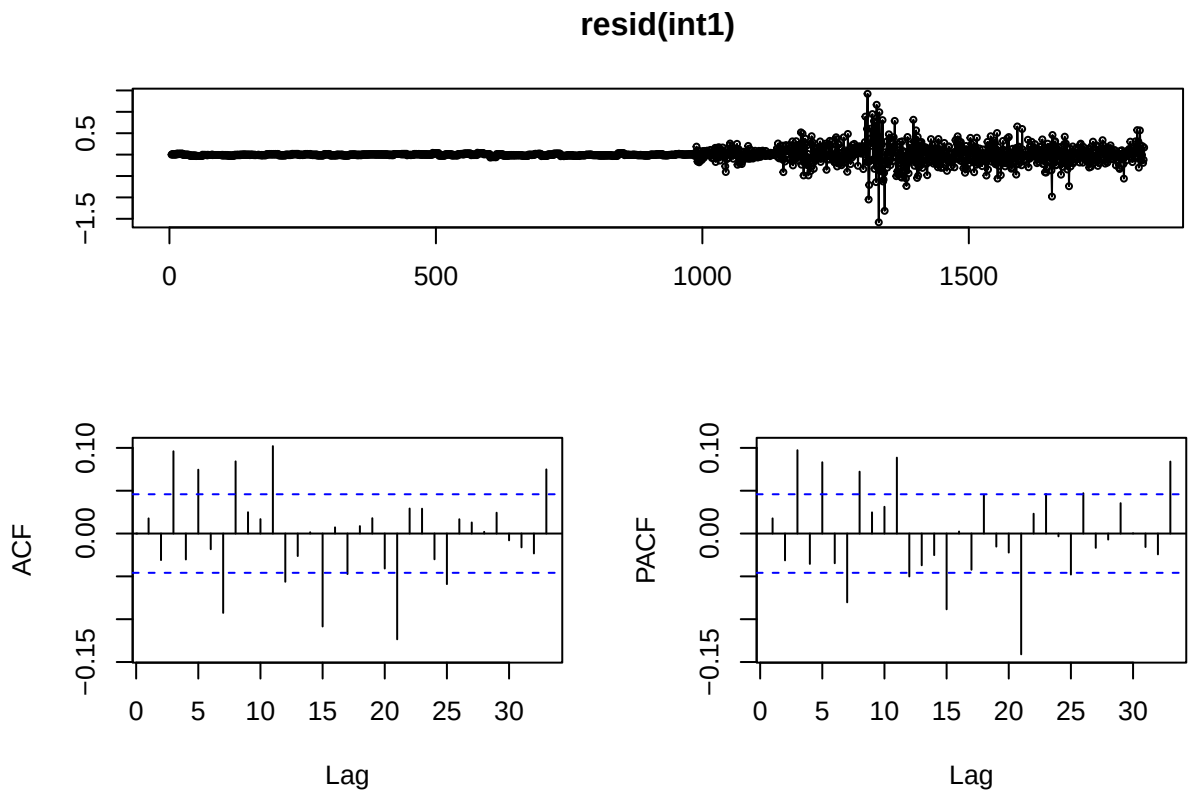
This model has a constant ts plot until somewhere around observation 1600, when things get a little more active. The ACF plot features little decline aside from one major outlier at lag 12, suggesting some sort of annual relationship. The PACF sees no significant lag.

```
tsdisplay(resid(earn2))
```



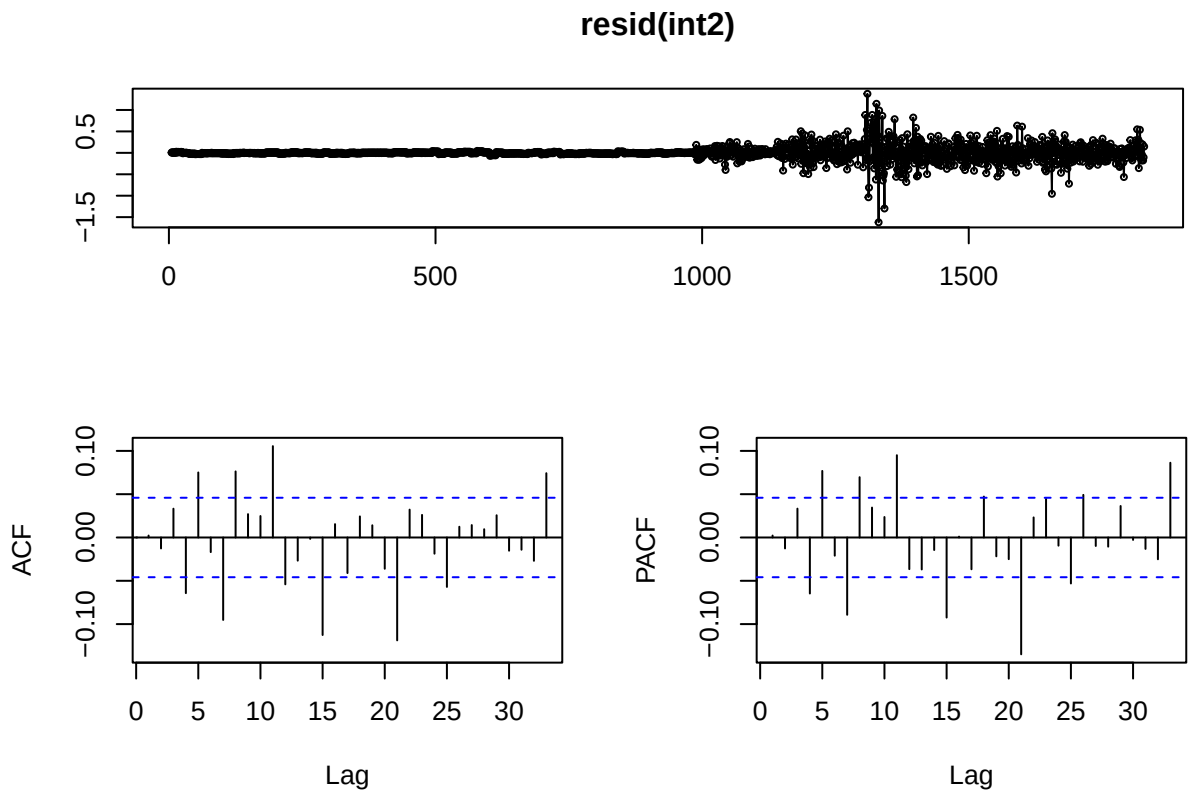
These plots are very similar to the AR(2) versions, with the only exception being even less lag from PACF.

```
tsdisplay(resid(int1))
```



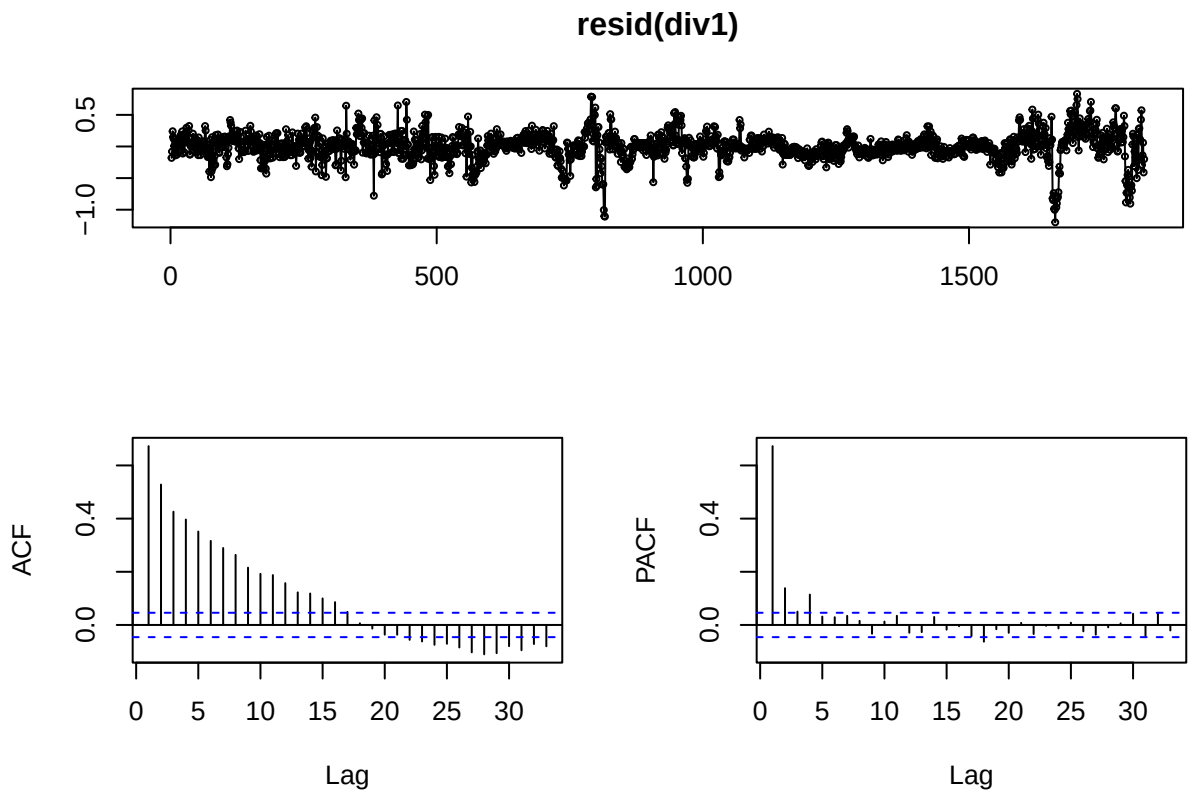
The first half of this model's ts plot is constant, but then it seems to start following a random walk. Much like the sp models, the ACF and PACF seem to follow seasonal patterns, but no obvious trends or drop offs to gain info from.

```
tsdisplay(resid(int2))
```



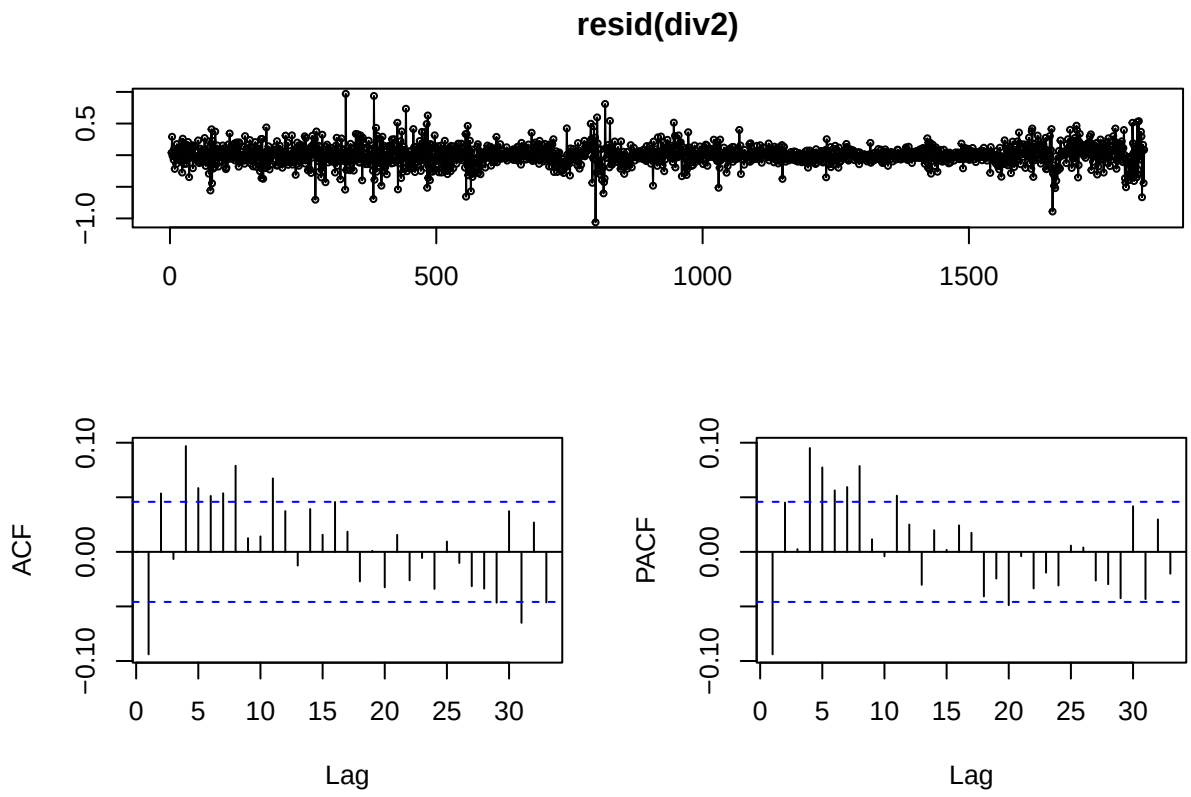
Same analysis as above

```
tsdisplay(resid(div1))
```



The ts display for the div AR(1) shows a good example of a random walk. The ACF has a slow decline towards the threshold, and the PACF features a sharp dropoff, which is ideal. This output would suggest a lag of 4 for future modeling.

```
tsdisplay(resid(div2))
```



Same analysis as above.

Training and Testing

```
train_size <- floor(2/3 * length(sp_ts))


train_ts <- sp_ts[1:train_size]
test_ts <- sp_ts[(train_size+1):length(sp_ts)]

train_earn <- earn_ts[1:train_size]
test_earn <- earn_ts[(train_size+1):length(earn_ts)]

train_int <- interest_ts[1:train_size]
test_int <- interest_ts[(train_size+1):length(interest_ts)]

train_div <- div_ts[1:train_size]
test_div <- div_ts[(train_size+1):length(div_ts)]



sp1_train <- dynlm(train_ts ~ L(train_ts, 1))
pred_sp1t <- predict(sp1, newdata = data.frame(lag(train_ts, 1)))
```

```
## Warning: 'newdata' had 1219 rows but variables found have 1829 rows
```

```
rmse_sp1t <- sqrt(mean((test_ts - pred_sp1t)^2, na.rm = TRUE))
```

```
## Warning in test_ts - pred_sp1t: longer object length is not a multiple of
## shorter object length
```

```
sp2_train <- dynlm(train_ts ~ L(train_ts, 1) + L(train_ts, 2))
pred_sp2t <- predict(sp2, newdata = data.frame(lag(train_ts, 1), lag(train_ts, 2)))
```

```
## Warning: 'newdata' had 1219 rows but variables found have 1829 rows
```

```
rmse_sp2t <- sqrt(mean((test_ts - pred_sp2t)^2, na.rm = TRUE))
```

```
## Warning in test_ts - pred_sp2t: longer object length is not a multiple of
## shorter object length
```

```
earn1_train <- dynlm(train_earn ~ L(train_earn, 1) + L(train_earn, 2))
pred_earn1t <- predict(earn1, newdata = data.frame(lag(train_earn, 1), lag(train_earn, 2)))
```

```
## Warning: 'newdata' had 1219 rows but variables found have 1829 rows
```

```
rmse_earn1t <- sqrt(mean((test_earn - pred_earn1t)^2, na.rm = TRUE))
```

```
## Warning in test_earn - pred_earn1t: longer object length is not a multiple of
## shorter object length
```

```
earn2_train <- dynlm(train_earn ~ L(train_earn, 1) + L(train_earn, 2) + L(train_earn, 3))
pred_earn2t <- predict(earn2, newdata = data.frame(lag(train_earn, 1), lag(train_earn, 2), lag(train_ea
```

```
## Warning: 'newdata' had 1219 rows but variables found have 1829 rows
```

```
rmse_earn2t <- sqrt(mean((test_earn - pred_earn2t)^2, na.rm = TRUE))
```

```
## Warning in test_earn - pred_earn2t: longer object length is not a multiple of
## shorter object length
```

```
int1_train <- dynlm(train_int ~ L(train_int, 1) + L(train_int, 2) + L(train_int, 3))
pred_int1t <- predict(int1, newdata = data.frame(lag(train_int, 1), lag(train_int, 2), lag(train_int, 3
```

```
## Warning: 'newdata' had 1219 rows but variables found have 1829 rows
```

```
rmse_int1t <- sqrt(mean((test_int - pred_int1t)^2, na.rm = TRUE))
```

```
## Warning in test_int - pred_int1t: longer object length is not a multiple of
## shorter object length
```

```
int2_train <- dynlm(train_int ~ L(train_int, 1) + L(train_int, 2) + L(train_int, 3)+ L(train_int, 4))
pred_int2t <- predict(int2, newdata = data.frame(lag(train_int, 1), lag(train_int, 2), lag(train_int, 3
```

```
## Warning: 'newdata' had 1219 rows but variables found have 1829 rows
```

```
rmse_int2t <- sqrt(mean((test_int - pred_int2t)^2, na.rm = TRUE))
```

```
## Warning in test_int - pred_int2t: longer object length is not a multiple of
## shorter object length
```

```
div1_train <- dynlm(train_div ~ L(train_div, 1))
pred_div1t <- predict(div1, newdata = data.frame(lag(train_div, 1)))
```

```
## Warning: 'newdata' had 1219 rows but variables found have 1829 rows
```

```
rmse_div1t <- sqrt(mean((test_div - pred_div1t)^2, na.rm = TRUE))
```

```
## Warning in test_div - pred_div1t: longer object length is not a multiple of
## shorter object length
```

```
div2_train <- dynlm(train_div ~ L(train_div, 1) + L(train_div, 2))
pred_div2t <- predict(div2, newdata = data.frame(lag(train_div, 1), lag(train_div, 2)))
```

```
## Warning: 'newdata' had 1219 rows but variables found have 1829 rows
```

```
rmse_div2t <- sqrt(mean((test_div - pred_div2t)^2, na.rm = TRUE))
```

```
## Warning in test_div - pred_div2t: longer object length is not a multiple of
## shorter object length
```

RMSE

```
rmse_sp1t
```

```
## [1] 1207.044
```

```
rmse_sp2t
```

```
## [1] 1207.032
```

```
rmse_earn1t
```

```
## [1] 56.06082
```

```
rmse_earn2t
```

```
## [1] 56.06171
```

```
rmse_int1t
```

```
## [1] 3.528202
```

```
rmse_int2t
```

```
## [1] 3.527982
```

```
rmse_div1t
```

```
## [1] 19.50353
```

```
rmse_div2t
```

```
## [1] 19.50963
```

Each pair of RMSE are alomst identical, but if we are only looking at the lowest value regardless of how close each pair is, then sp2, earn1, int2, and div1 are the best models by rmse.

AIC

```
AIC(sp1, sp2)
```

```
## Warning in AIC.default(sp1, sp2): models are not all fitted to the same number
## of observations
```

```
##     df      AIC
## sp1  3 17834.07
## sp2  4 17807.81
```

```
AIC(earn1, earn2)
```

```
## Warning in AIC.default(earn1, earn2): models are not all fitted to the same
## number of observations
```

```
##       df      AIC
## earn1  4 4845.559
## earn2  5 4840.691
```

```
AIC(int1, int2)
```

```
## Warning in AIC.default(int1, int2): models are not all fitted to the same
## number of observations
```

```
##      df       AIC
## int1  5 -1349.991
## int2  6 -1359.692
```

```
AIC(div1, div2)
```

```
## Warning in AIC.default(div1, div2): models are not all fitted to the same
## number of observations
```

```
##      df        AIC
## div1  3  -417.5042
## div2  4 -1517.0075
```

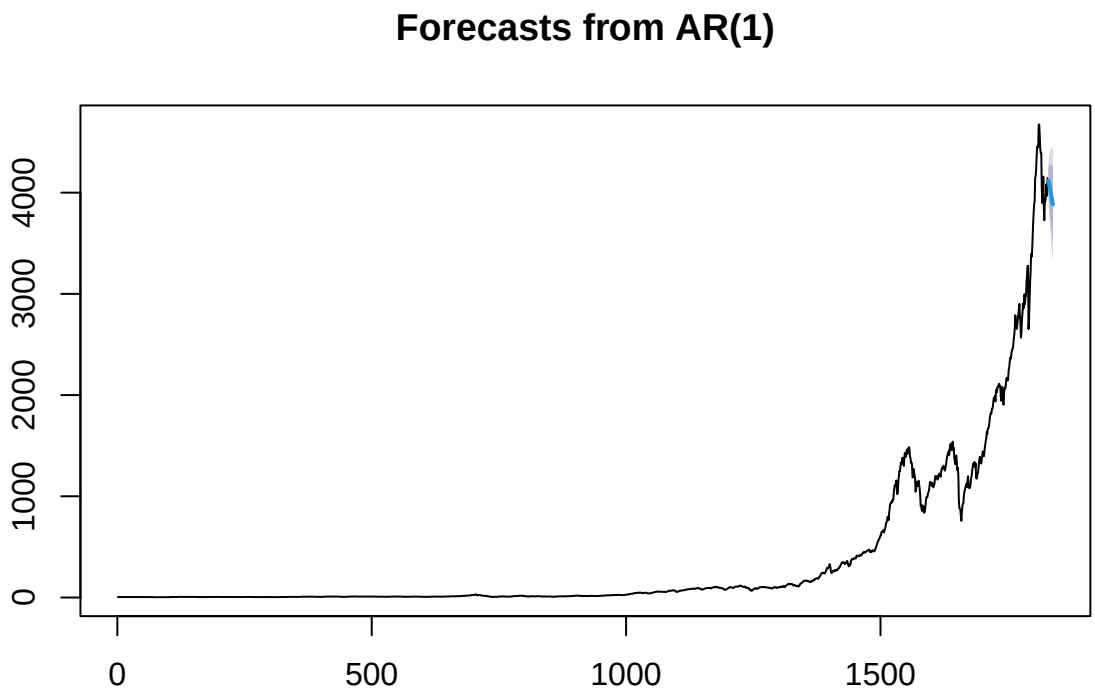
sp2 (the AR(2) model) has a lower AIC than sp1, earn2 (AR(3) model) has a lower AIC than earn1, int2 (AR(4)) has a lower AIC than int1, and div2 (AR(2)) has a significantly lower AIC than div1. When analyzing AIC, the goal is to be lower, so the superior models based on AIC are sp2, earn2, int2, and div2.

Forecasting

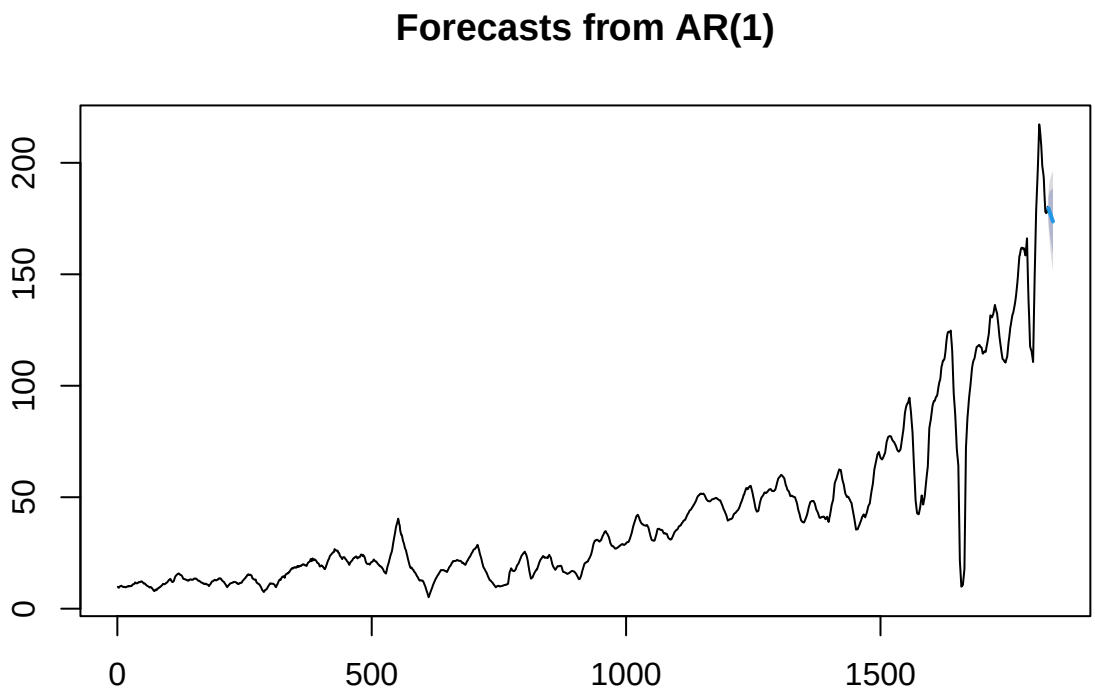
```
ar_sp500_1t <- ar(sp_ts, order.max = 1, method = "yw")
ar_sp500_2t <- ar(sp_ts, order.max = 2, method = "yw")
ar_earn_1t <- ar(earn_ts, order.max = 1, method = "yw")
ar_earn_2t <- ar(earn_ts, order.max = 3, method = "yw")
ar_int_1t <- ar(interest_ts, order.max = 3, method = "yw")
ar_int_2t <- ar(interest_ts, order.max = 4, method = "yw")
ar_div_1t <- ar(div_ts, order.max = 1, method = "yw")
ar_div_2t <- ar(div_ts, order.max = 2, method = "yw")
```

```
f1 <- forecast(ar_sp500_1t, h = 10)
f2 <- forecast(ar_sp500_2t, h = 10)
f3 <- forecast(ar_earn_1t, h = 10)
f4 <- forecast(ar_earn_2t, h = 10)
f5 <- forecast(ar_int_1t, h = 10)
f6 <- forecast(ar_int_2t, h = 10)
f7 <- forecast(ar_div_1t, h = 10)
f8 <- forecast(ar_div_2t, h = 10)
```

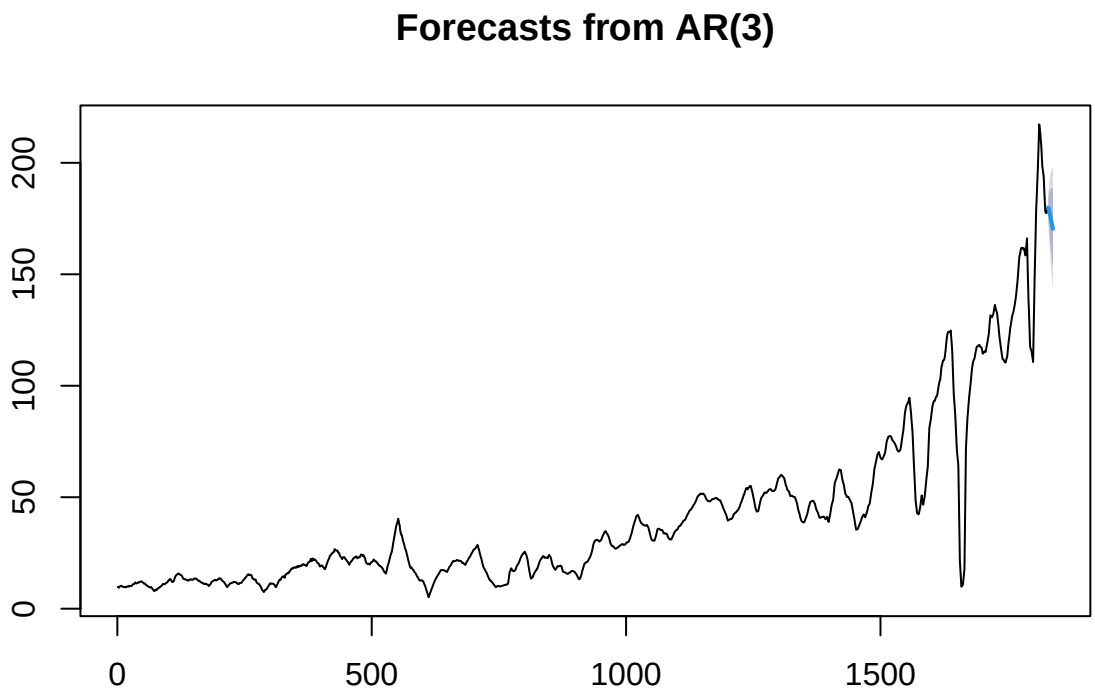
```
plot(f1)
plot(f2)
```



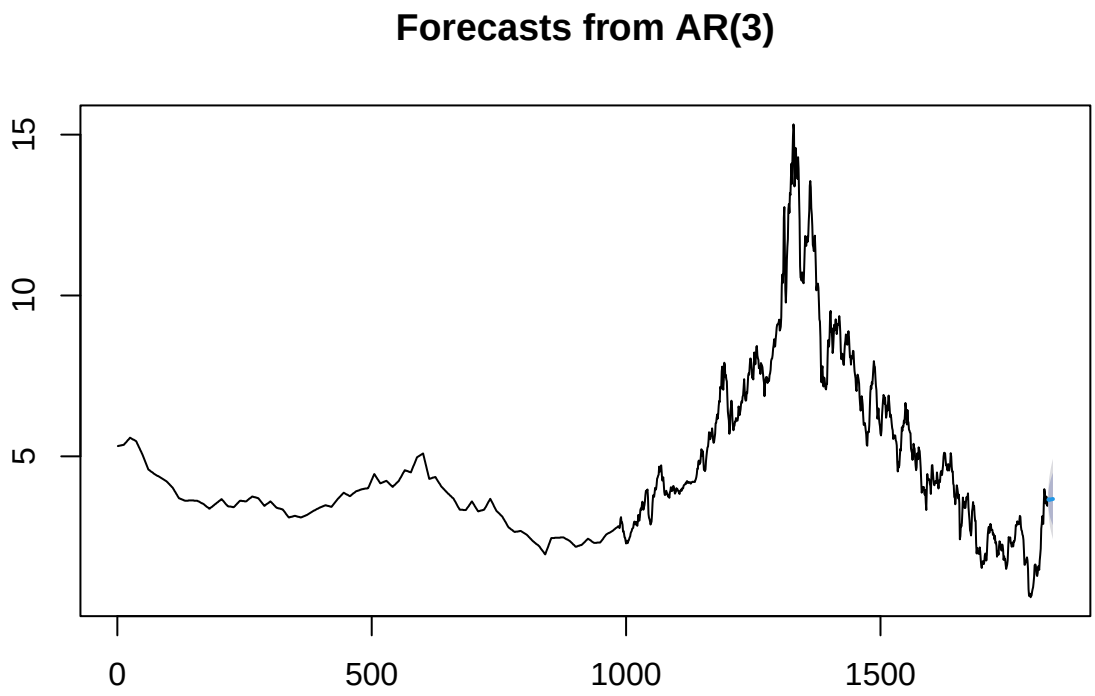

```
plot(f3)
```



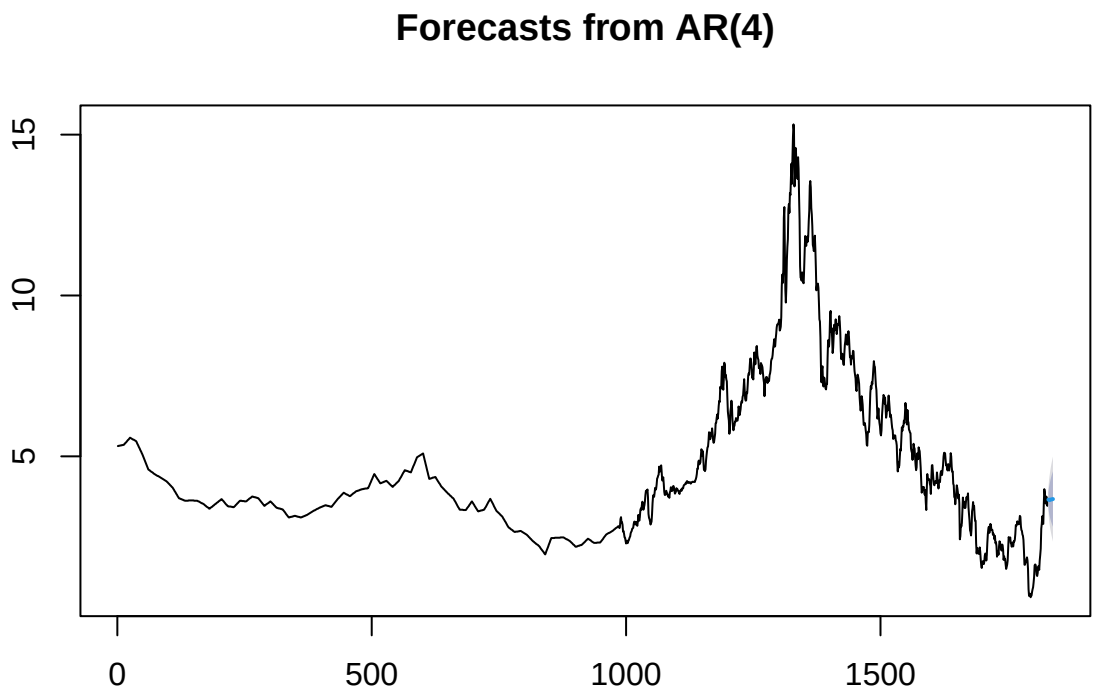

```
plot(f4)
```



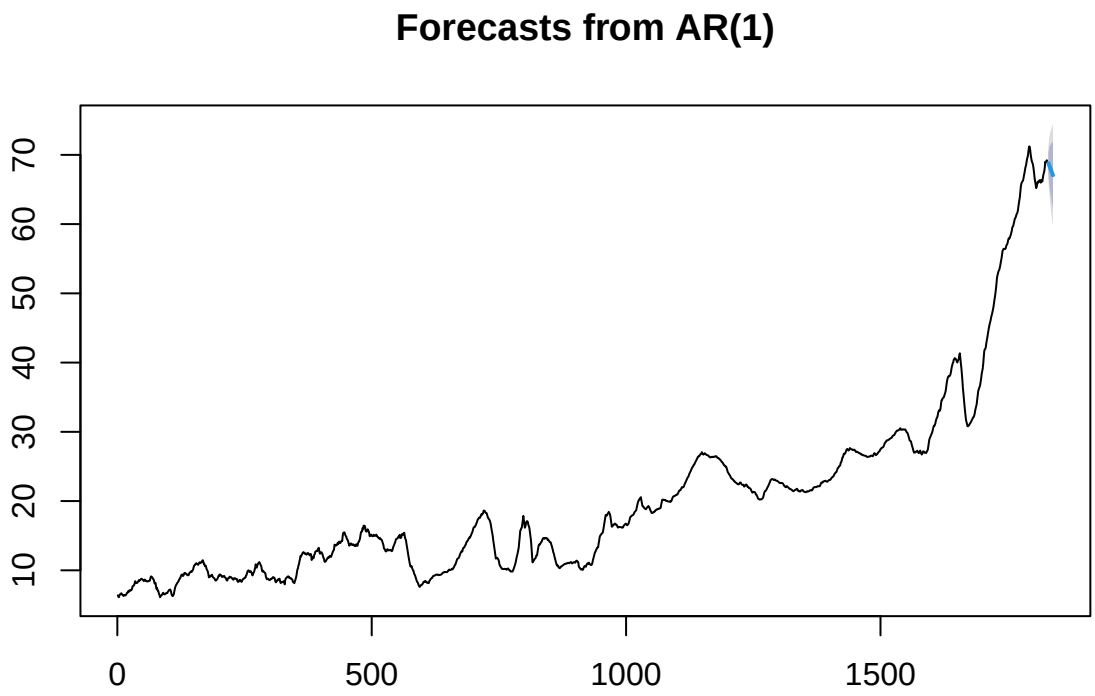

```
plot(f5)
```




```
plot(f6)
```




```
plot(f7)
plot(f8)
```



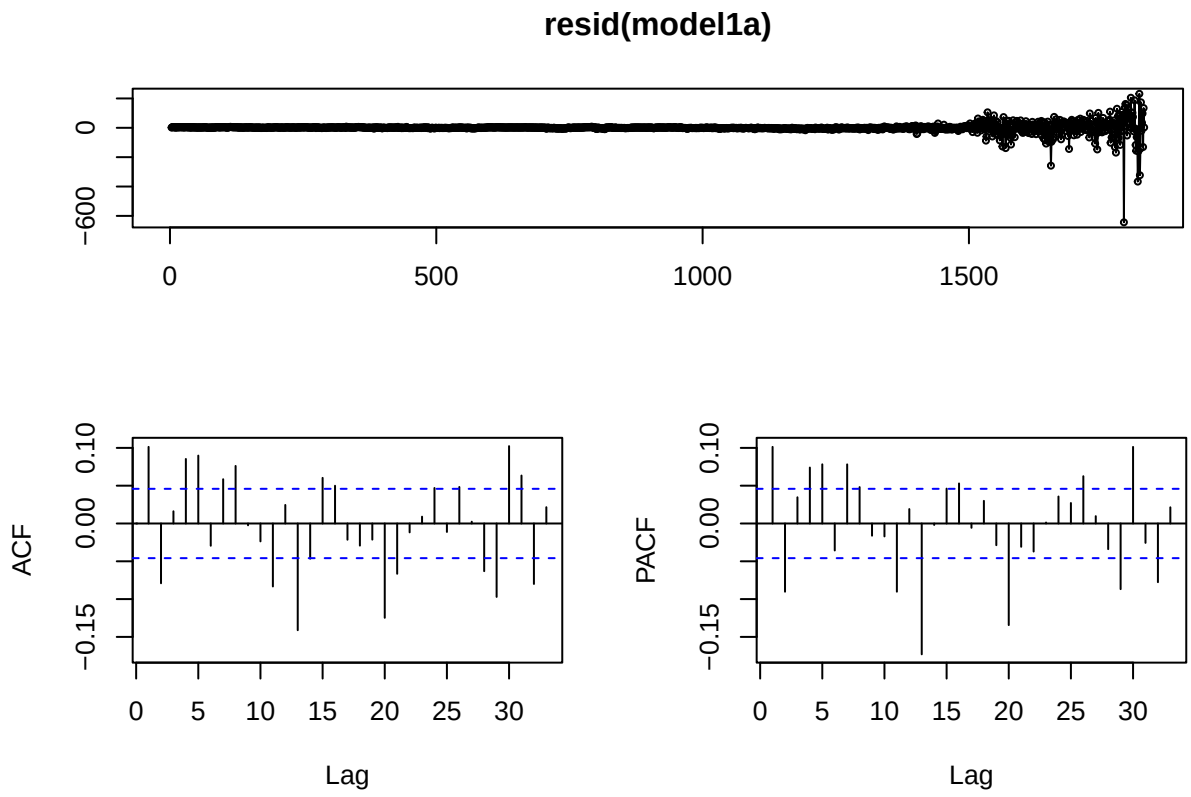
Question 4

Modeling

```
model1a <- dynlm(sp_ts ~ L(sp_ts, 1) + L(div_ts, 1) + L(div_ts, 2)) #ARDL(1, 2)

model1b <- dynlm(sp_ts ~ L(sp_ts, 1) + L(interest_ts, 1) + L(interest_ts, 2)+ L(interest_ts, 3))
#ARDL(1, 3)



model2a <- dynlm(earn_ts ~ L(earn_ts, 1) + L(earn_ts, 2) + L(div_ts, 1) + L(div_ts, 2))
#ARDL(2, 2)
model2b <- dynlm(earn_ts ~ L(earn_ts, 1) + L(interest_ts, 1))
#ARDL(1, 1)
```

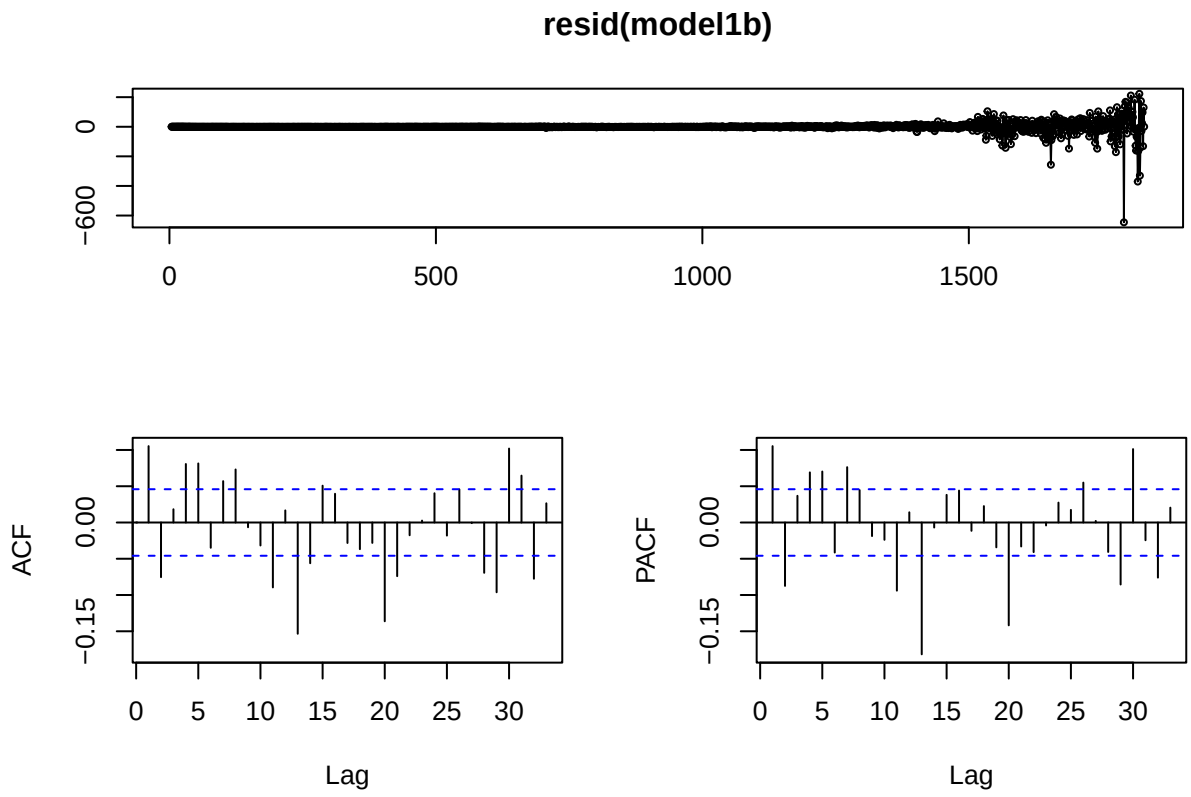
Residuals

```
tsdisplay(resid(model1a))
```



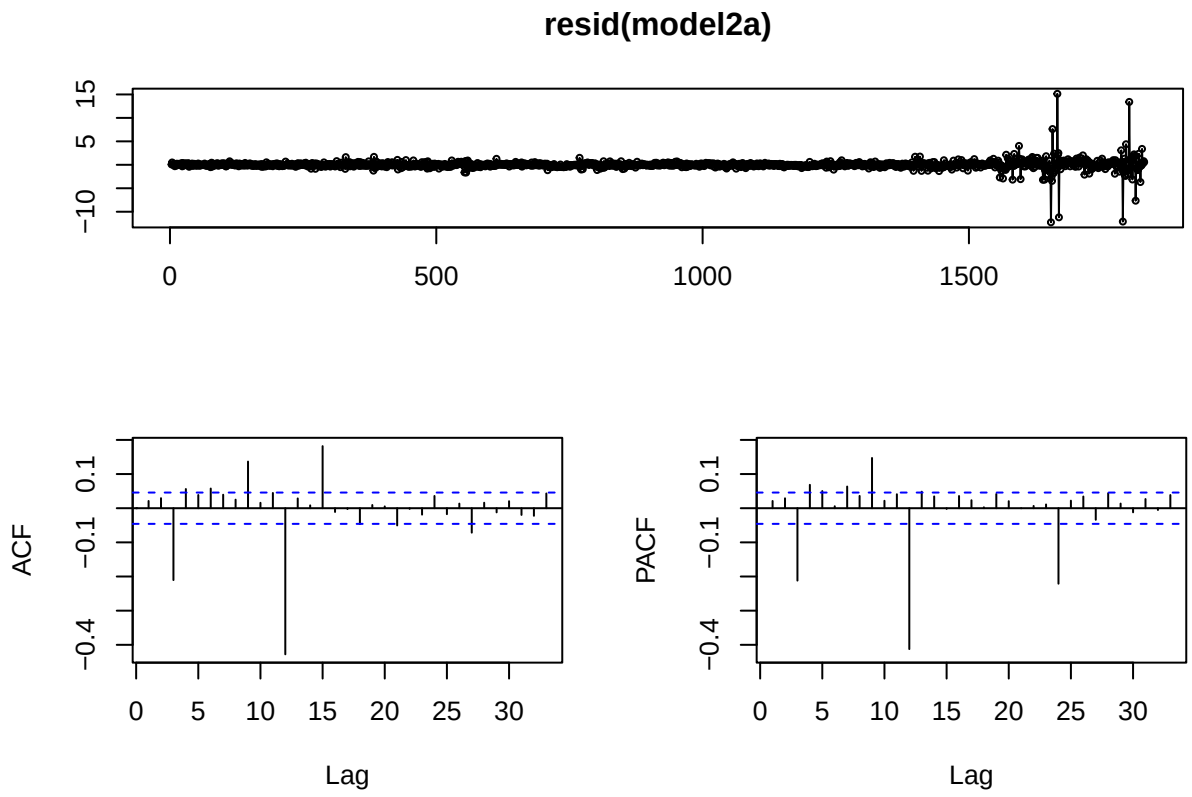
This model above features a constant ts plot at 0 until right before observation 1500, where we see a minor random walk pattern. It would be ideal to see more of this. The ACF plot does decreace slowly at the beginning, but it never really stays within the threshold consistently. This lack of a steady decrease makes it harder to judge our PACF graph, which seems to show a steep decline after lag 2.

```
tsdisplay(resid(model1b))
```



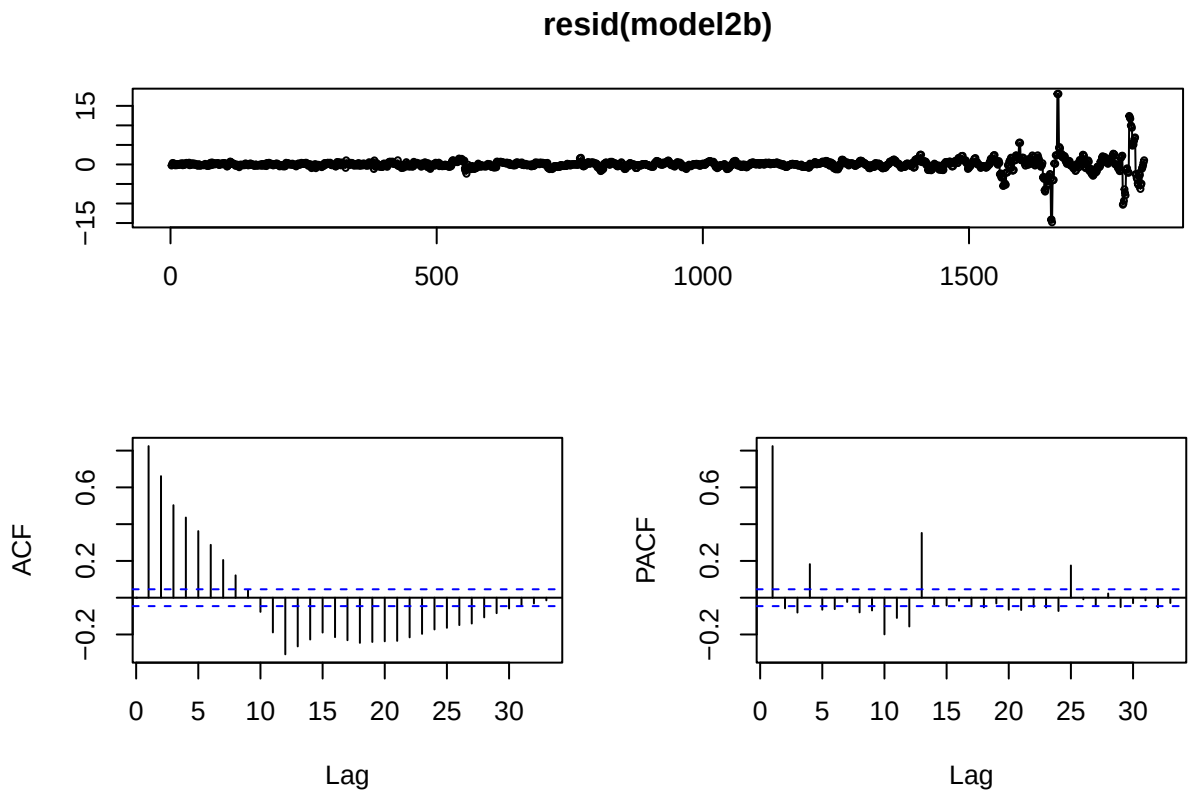
This model is almost identical to model1a despite removing the Dividends variable and adding in interest. The graphs follow the same analysis as model1a, but if switching the variables made so little difference, it makes me think that SP is better predicted with just an AR model.

```
tsdisplay(resid(model2a))
```



With model2, we see the residuals have more activity building up towards some sharper points around observation 1630, but less of an overall random walk compared to the residuals of model1a. The ACF and PACF plots are almost identical. Neither diminish slowly, and there aren't any steep drop offs.

```
tsdisplay(resid(model2b))
```



The residuals of Model2b paint a more interesting picture than the other three models. The ts plot is very similar to model2a, with a little more random walk towards the end, but the ACF here does show a gradual decline. This means that if the PACF has a sharp drop (it does), then lags can be interpreted. In this case, there is one immediate lag, with a flare up at lag 4 (quarterly patterns) and lag 13 (almost annual patterns).

RMSE

```
tsdata <- ts(sandp)

train_size <- floor(2/3 * nrow(tsdata))
train_data <- tsdata[1:train_size, ]
test_data <- tsdata[(train_size + 1):nrow(tsdata), ]
```

```
tmodel1a <- ardl(SP500 ~ Div, data = train_data, order = c(1, 2))
tmodel1b <- ardl(SP500 ~ Interest, data = train_data, order = c(1, 3))

tmodel2a <- ardl(Earn ~ Div, data = train_data, order = c(2, 2))
tmodel2b <- ardl(Earn ~ Interest, data = train_data, order = c(1, 2))
```

```
predictions1a <- predict(tmodel1a, newdata = test_data)
```

```
## Error in model.frame.default(Terms, newdata, na.action = na.action, xlev = object$xlevels): 'data' mu
```

```
predictions1b <- predict(tmodel1b, newdata = test_data)
```

```
## Error in model.frame.default(Terms, newdata, na.action = na.action, xlev = object$xlevels): 'data' mu
```

```
predictions2a <- predict(tmodel2a, newdata = test_data)
```

```
## Error in model.frame.default(Terms, newdata, na.action = na.action, xlev = object$xlevels): 'data' mu
```

```
predictions2b <- predict(tmodel2b, newdata = test_data)
```

```
## Error in model.frame.default(Terms, newdata, na.action = na.action, xlev = object$xlevels): 'data' mu
```

I was unable to find a solution to this part of the project. No function L was in my code, yet this error kept popping up.

AIC

```
AIC(model1a, model1b)
```

```
## Warning in AIC.default(model1a, model1b): models are not all fitted to the same
## number of observations
```

```
##         df      AIC
## model1a  5 17813.09
## model1b  6 17817.63
```

```
AIC(model2a, model2b)
```

```
## Warning in AIC.default(model2a, model2b): models are not all fitted to the same
## number of observations
```

```
##         df      AIC
## model2a  6 4779.375
## model2b  4 6934.156
```

The AIC for Models 1a and 1b are almost identical, with 1a being slightly lower. Models 2a and 2b are much further apart. Despite model2b having a much more standard residual plot, model2a has an AIC that is way lower.

Forecasting

```
uecm_tmodel1a <- uecm(tmodel1a)
uecm_tmodel1b <- uecm(tmodel1b)
uecm_tmodel2a <- uecm(tmodel2a)
uecm_tmodel2b <- uecm(tmodel2b)


forecast_result1a <- forecast(uecm_tmodel1a, h = 10)
```

```
## Error in L(SP500, 1): could not find function "L"
```

```
forecast_result1b <- forecast(uecm_tmodel1b, h = 10)
```

```
## Error in L(SP500, 1): could not find function "L"
```

```
forecast_result2a <- forecast(uecm_tmodel2a, h = 10)
```

```
## Error in L(Earn, 1): could not find function "L"
```

```
forecast_result2b <- forecast(uecm_tmodel2b, h = 10)
```

```
## Error in L(Earn, 1): could not find function "L"
```

```
autoplot(forecast_result1a)
```

```
## Error in autoplot(forecast_result1a): object 'forecast_result1a' not found
```

```
autoplot(forecast_result1b)
```

```
## Error in autoplot(forecast_result1b): object 'forecast_result1b' not found
```

```
autoplot(forecast_result2a)
```

```
## Error in autoplot(forecast_result2a): object 'forecast_result2a' not found
```

```
autoplot(forecast_result2b)
```

```
## Error in autoplot(forecast_result2b): object 'forecast_result2b' not found
```

Question 5

Variable Analysis

```
adf.test(sp_ts)
```

```
## Warning in adf.test(sp_ts): p-value greater than printed p-value

##
##  Augmented Dickey-Fuller Test
##
## data:  sp_ts
## Dickey-Fuller = 2.7921, Lag order = 12, p-value = 0.99
## alternative hypothesis: stationary
```

```r
adf.test(earn_ts)
```

```
## Warning in adf.test(earn_ts): p-value greater than printed p-value

##
##  Augmented Dickey-Fuller Test
##
## data:  earn_ts
## Dickey-Fuller = -0.26603, Lag order = 12, p-value = 0.99
## alternative hypothesis: stationary
```

```r
dsp <- diff(sp_ts)
dearn <- diff(earn_ts)
```

```r
dsp <- diff(sp_ts)
dearn <- diff(earn_ts)

var_data <- cbind(dsp, dearn)
```

```r
lag_selection <- VARselect(var_data, lag.max = 5, type = "both")
print(lag_selection)
```

```
## $selection
## AIC(n)  HQ(n)  SC(n) FPE(n)
##      5      5      5      5
##
## $criteria
##                1          2          3          4          5
## AIC(n)   6.666474   6.645132   6.625892   6.581093   6.554681
## HQ(n)    6.675391   6.658507   6.643726   6.603386   6.581432
## SC(n)    6.690646   6.681390   6.674236   6.641524   6.627197
## FPE(n) 785.620767 769.031285 754.376572 721.327491 702.525108
```

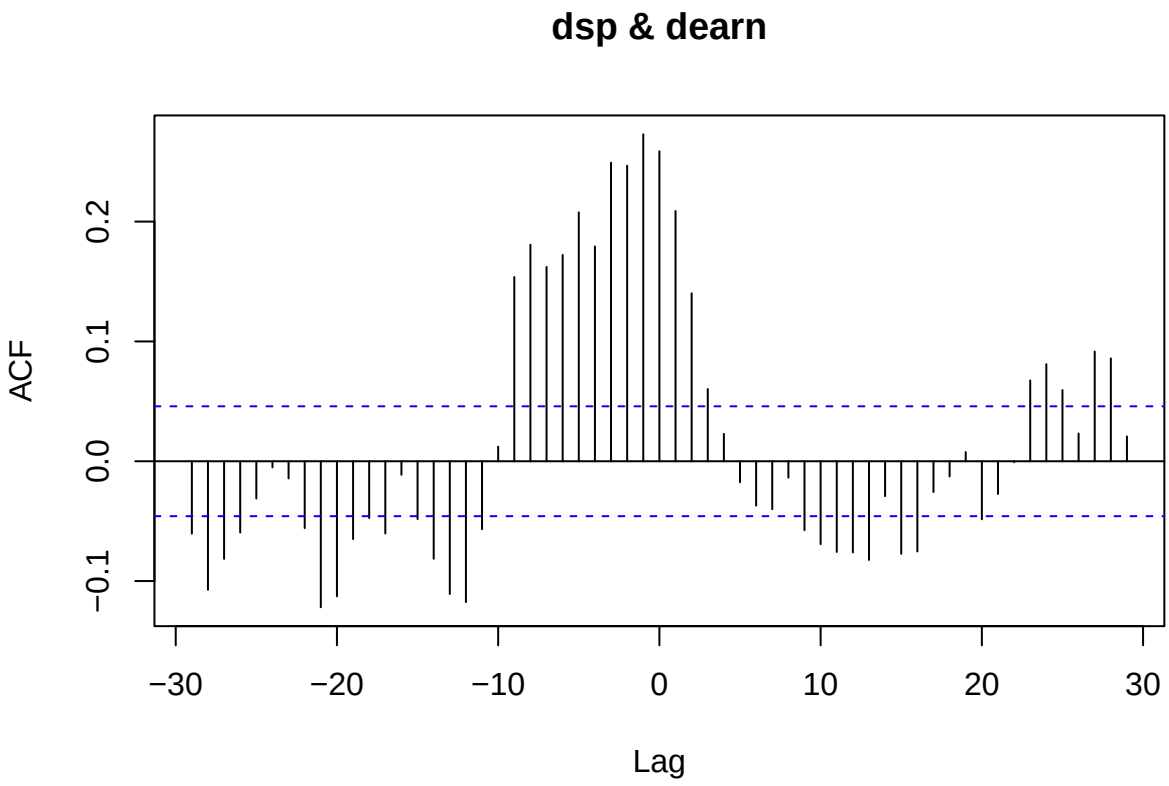
Based on the previous lines of code, var(5) will be optimal for this data.

Modeling

```r
var_model <- VAR(var_data, p = 5, type = "both")
```

CCF Test

```r
ccf(dsp, dearn)
```



According to the CCF plot, the most significant lags take place around h = -9 and maybe h = -11. It can be tough to gain exact values from the plot. This would mean that dsp affects dearn around 10 periods later. Looking at the other side of the plot, there are drop offs around h = 2 and h = 8, which would mean that dearn affects dsp 2 and 8 lags after.

The main purpose of this graph is to tell us roughly where significant lags take place by looking at the steepest falls throughout the plot. In that sense, it is very similar to a PACF plot.

Granger Casuality Test

```
grangertest(dsp ~ dearn, order = 5)
```

```
## Granger causality test
##
## Model 1: dsp ~ Lags(dsp, 1:5) + Lags(dearn, 1:5)
## Model 2: dsp ~ Lags(dsp, 1:5)
##   Res.Df Df      F    Pr(>F)
## 1   1812
## 2   1817 -5 21.433 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

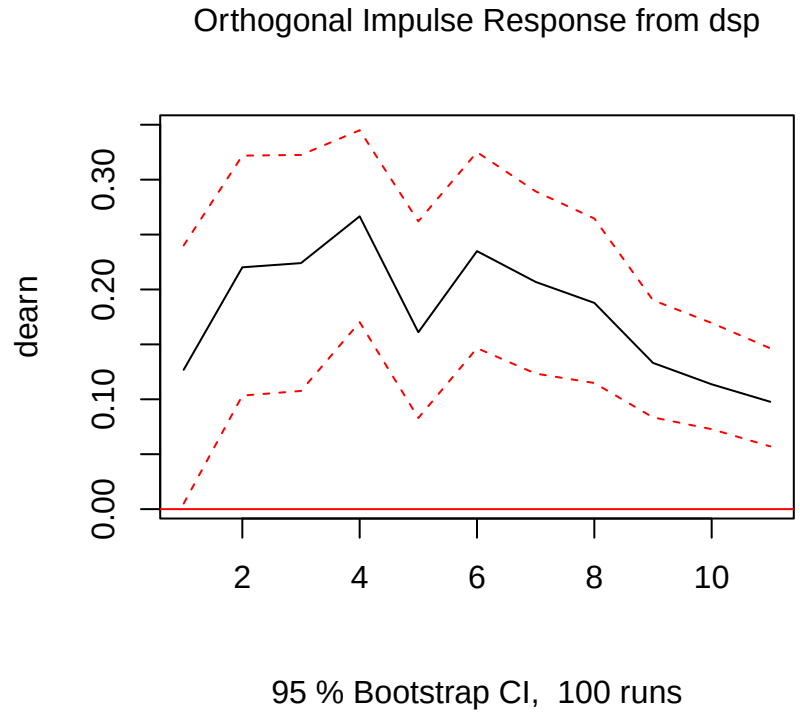
```
grangertest(dearn ~ dsp, order = 5)
```

```
## Granger causality test
##
## Model 1: dearn ~ Lags(dearn, 1:5) + Lags(dsp, 1:5)
## Model 2: dearn ~ Lags(dearn, 1:5)
##   Res.Df Df      F    Pr(>F)
## 1   1812
## 2   1817 -5 17.244 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The highly significant p values across both tests tells us that both variables influence each other. There is casualty between variables. In other words, it can be said that each variable may cause each other. Highly connected variables often make for better VAR models.

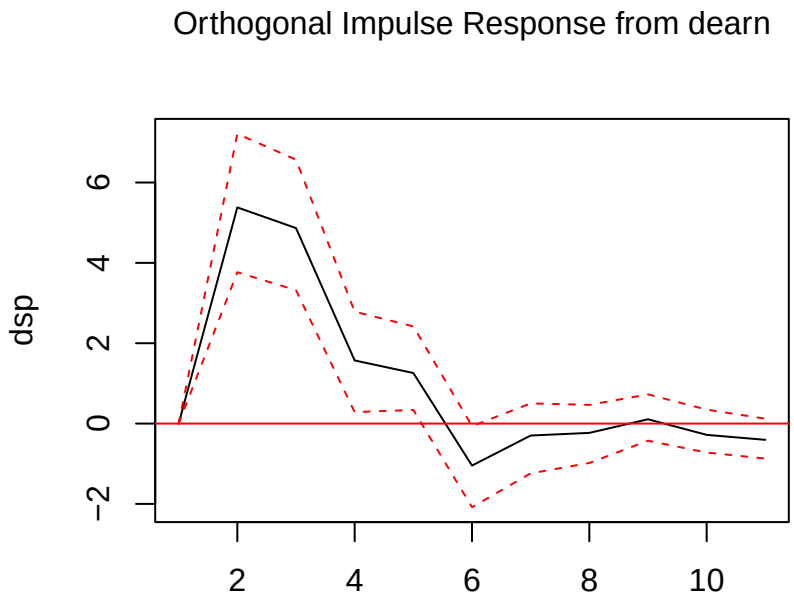
IRFs

```
irf <- irf(var_model, impulse = "dsp", response = "dearn", n.ahead = 10, boot = TRUE)
plot(irf)
```



This test looks at what happens to dEarn after a shock to dSP. For 8 periods (months, in our case), Earnings are affected by a sharp change in the stock market, but then around months 9 and 10 things usually go back to normal. In short, it takes most of a year for the dEarn variable to recover from a large change in dSP, and even then we don't know exactly when it goes back to 0% effect.

```r
irf2 <- irf(var_model, impulse = "dearn", response = "dsp", n.ahead = 10, boot = TRUE)
plot(irf2)
```

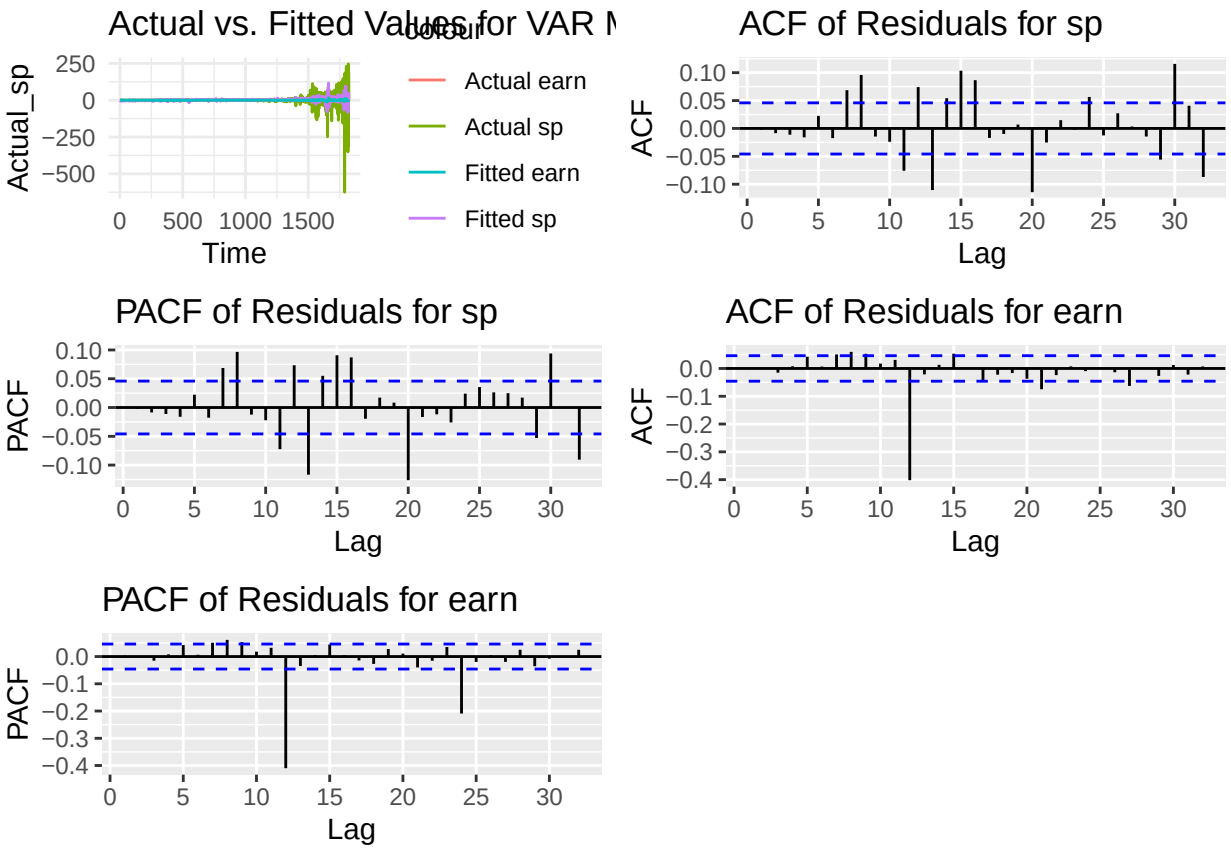


This IRF plot is the reverse of the prior one. It shows how the Stock Market reacts to a large shift in Earnings. When Earnings are shocked, the stock market goes crazy for about 4 periods (months, in our case), before returning to normal. It returns to normal much faster than dEarn does in the last example, but the affect on dSP in IRF2 is much more significant (7 vs 0.35).

All Inclusive Plot

```r
var_data <- as.data.frame(var_data)

var_data <- var_data[c(1:1823), ]

# Extract fitted values from VAR model
fitted_values <- fitted(var_model)

# Extract residuals
residuals_var <- residuals(var_model)

# Create a dataframe for actual vs fitted plot
df_plot <- data.frame(
  Time = 1:nrow(var_data),
  Actual_sp = var_data$dsp,
```

```
  Fitted_sp = fitted_values[, 1],
  Actual_earn = var_data$dearn,
  Fitted_earn = fitted_values[, 2]
)

# Actual vs. Fitted plot
p1 <- ggplot(df_plot, aes(x = Time)) +
  geom_line(aes(y = Actual_sp, color = "Actual sp")) +
  geom_line(aes(y = Fitted_sp, color = "Fitted sp")) +
  geom_line(aes(y = Actual_earn, color = "Actual earn")) +
  geom_line(aes(y = Fitted_earn, color = "Fitted earn")) +
  labs(title = "Actual vs. Fitted Values for VAR Model") +
  theme_minimal()

# ACF and PACF plots for residuals
p2 <- ggAcf(residuals_var[, 1]) + ggtitle("ACF of Residuals for sp")
p3 <- ggPacf(residuals_var[, 1]) + ggtitle("PACF of Residuals for sp")
p4 <- ggAcf(residuals_var[, 2]) + ggtitle("ACF of Residuals for earn")
p5 <- ggPacf(residuals_var[, 2]) + ggtitle("PACF of Residuals for earn")

# Arrange all plots together
grid.arrange(p1, p2, p3, p4, p5, ncol = 2)
```



The set of graphs above display the actual vs. fitted values for both response variables, along with their ACF and PACF plots. Not too much can be gleaned from the actual vs. fitted, except that the fitted variables tend to be more contained, but the ACF and PACF plots are more insightful. The sp plots seem to gradually

increase, which tends to mean that lags cannot be predicted from the plots themselves and more investigation is needed. What stood out to me in the Earnings plots was the large spike at 12. Despite both starting out slow, which makes the results harder to interpret, there definitely seems to be correlation between annual earnings.

RMSE

```
fitted_values <- fitted(var_model)
actual_values <- var_data[6:nrow(var_data), ]

rmse_dsp <- sqrt(mean((actual_values[,1] - fitted_values[,1])^2))
```

```
## Warning in actual_values[, 1] - fitted_values[, 1]: longer object length is not
## a multiple of shorter object length
```

```
rmse_dearn <- sqrt(mean((actual_values[,2] - fitted_values[,2])^2))
```

```
## Warning in actual_values[, 2] - fitted_values[, 2]: longer object length is not
## a multiple of shorter object length
```

```
rmse_dsp
```

```
## [1] 29.9141
```

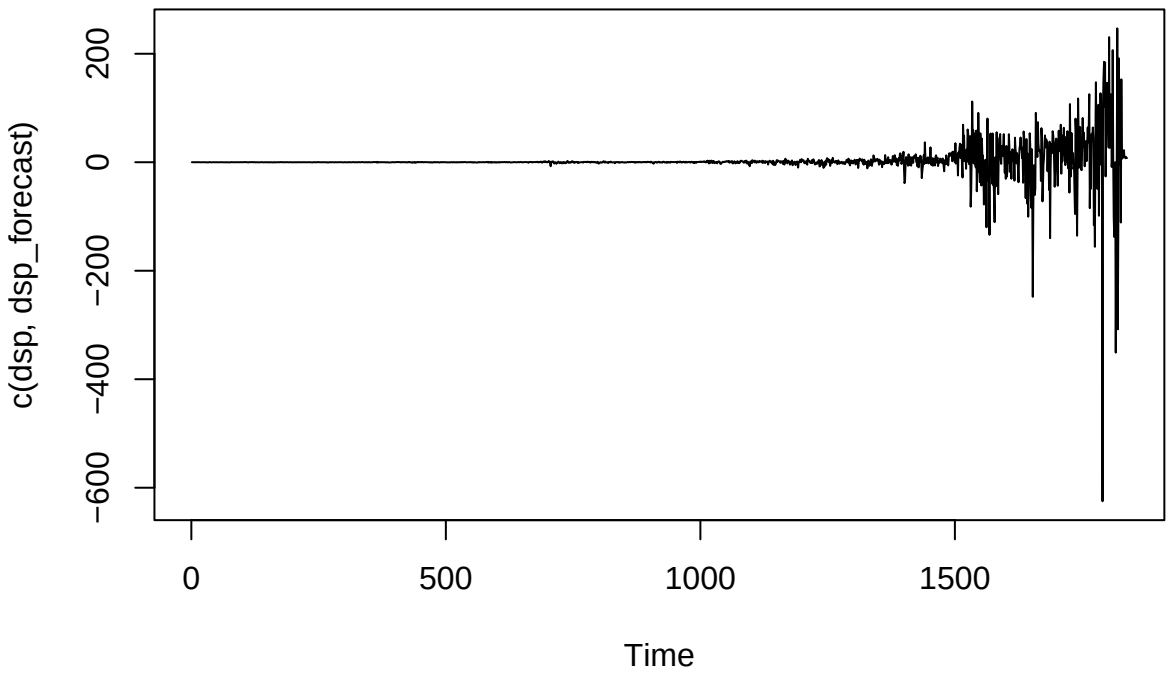
```
rmse_dearn
```

```
## [1] 0.8702974
```

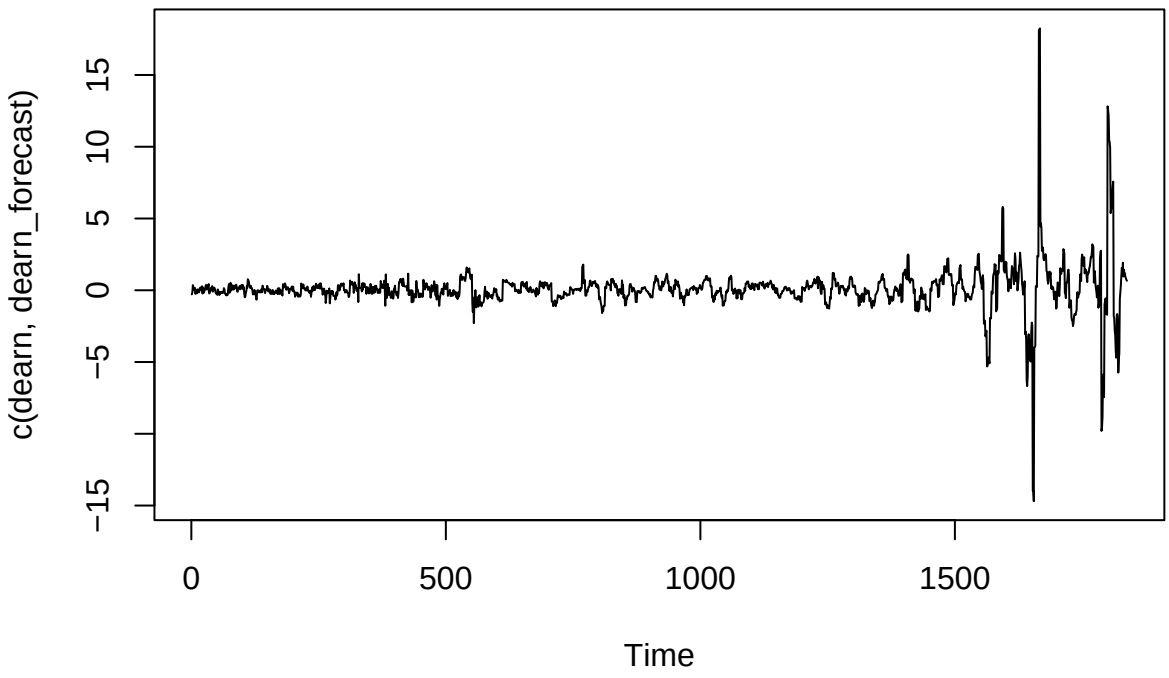
Both dSP and dEarn have extremely small RMSE in the VAR model. The dEarn value is especially small, implying that the model for our y2t is better than our model for y1t even though both seem to be functioning very well.

Forecast

```
var_forecast <- predict(var_model, n.ahead = 10)


dsp_forecast <- var_forecast$fcst$dsp[, 1]
dearn_forecast <- var_forecast$fcst$dearn[, 1]


ts.plot(c(dsp, dsp_forecast), col=c("black", "red"))
```



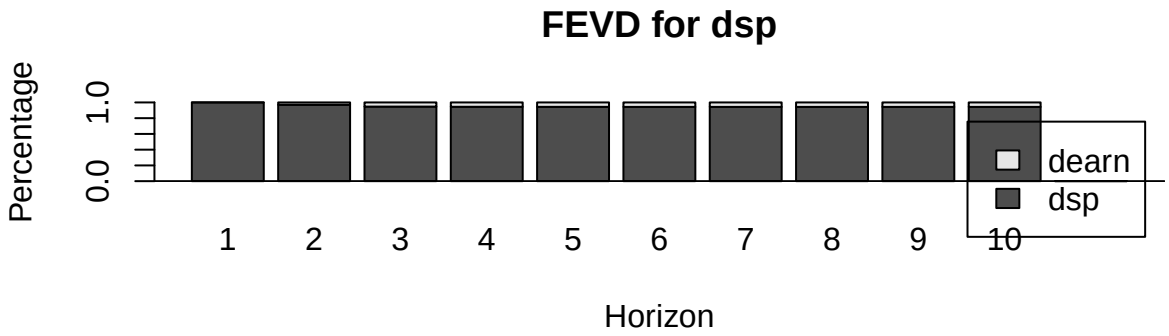

```
ts.plot(c(dearn, dearn_forecast), col=c("black", "blue"))
```

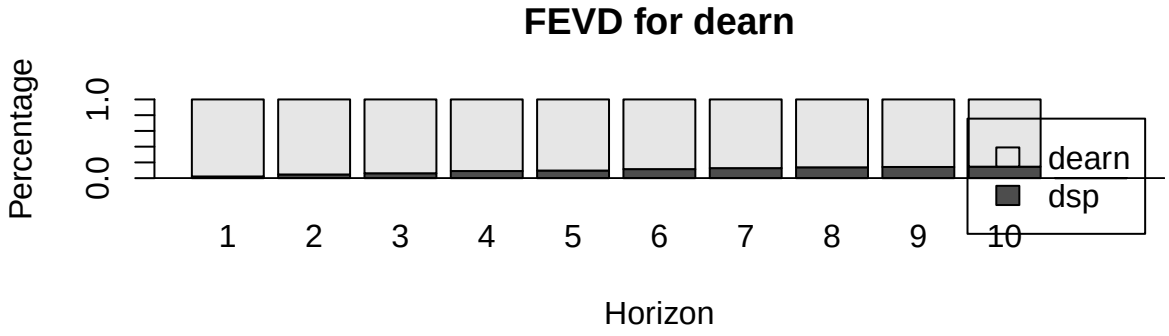


To keep these results in line with other parts of the project, I chose to predict n=10 steps ahead.

FEVD

```
fevd_model <- fevd(var_model, n.ahead = 10)
plot(fevd_model)
```

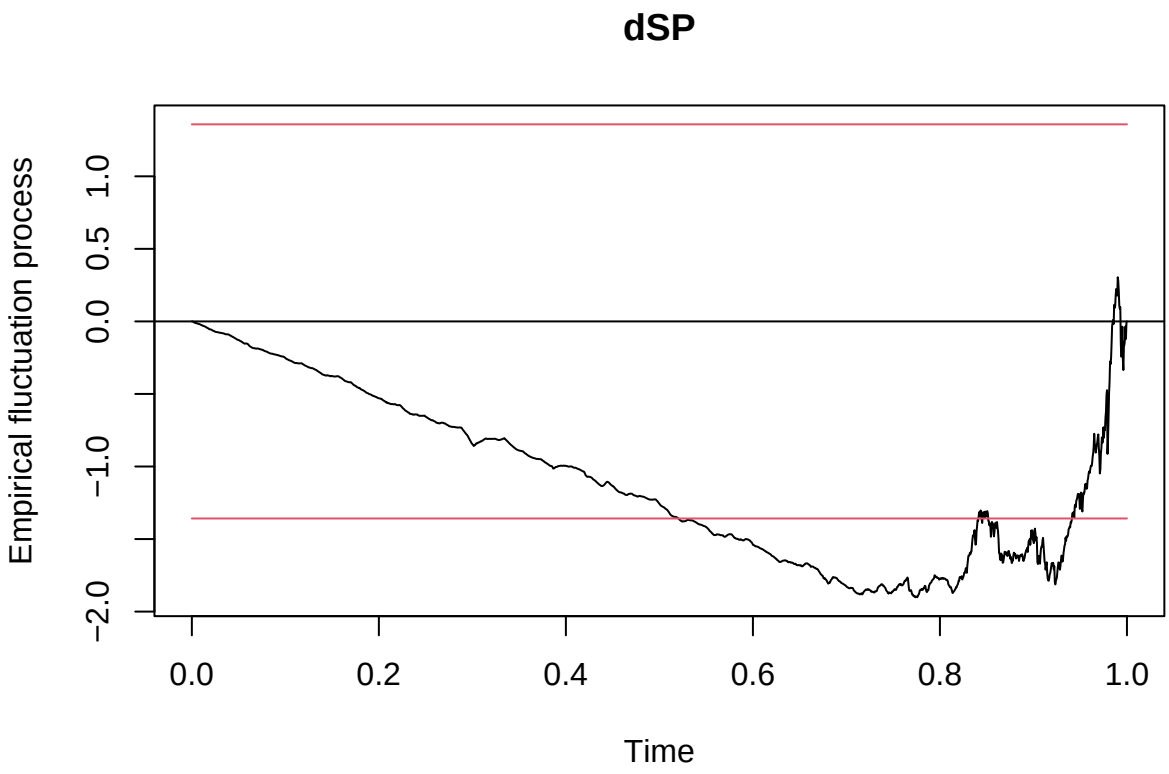




The FEVD test shows how much of a variables variance is caused by itself. By the looks of this output, very close to 100% of dSP's variance is caused by dSP, whereas about 90% of the dEarn variance is caused by dEarn. This is another way to visualize how much control each variable has on each other, and just how closely they may be related.

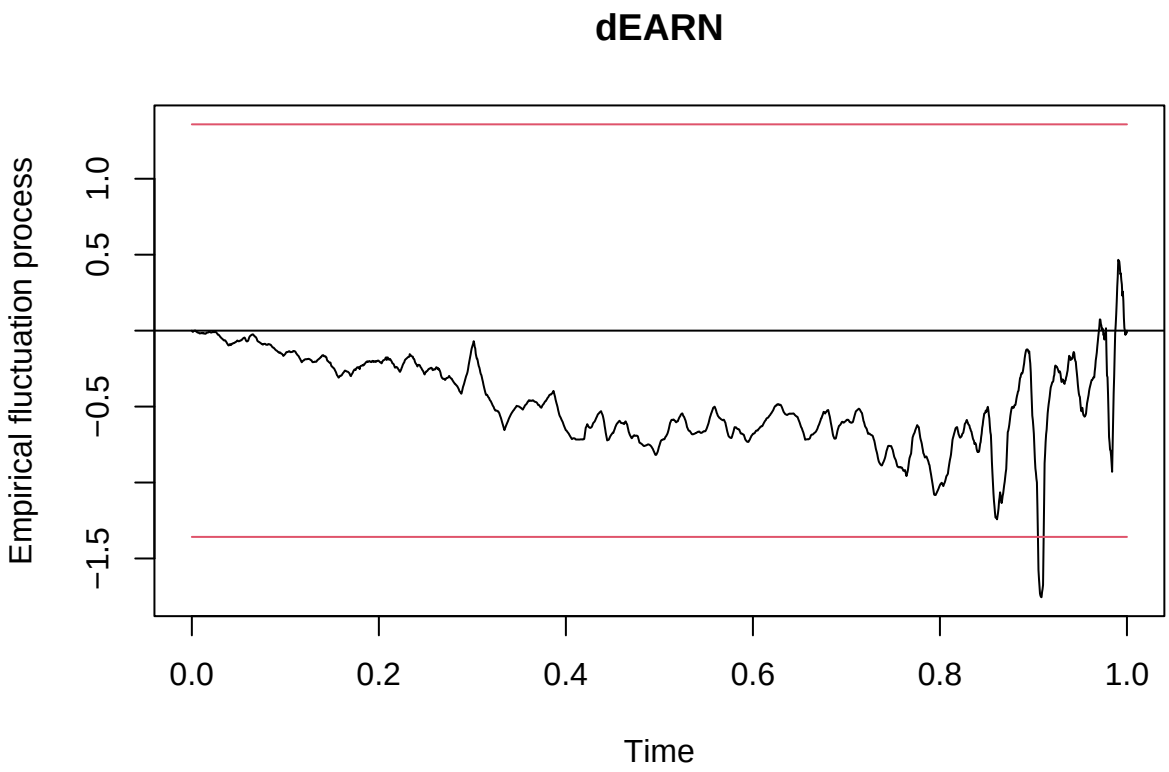
CUSUM

```
library(strucchange)

cusum_test_dsp <- efp(dsp ~ dearn, data = as.data.frame(var_data), type = "OLS-CUSUM")
cusum_test_dearn <- efp(dearn ~ dsp, data = as.data.frame(var_data), type = "OLS-CUSUM")

plot(cusum_test_dsp, main = "dSP")
```



This test is used to tell whether a variable has inherently changed throughout the tracking. It judges this by testing the cumulative sum, and then checking whether the line has crossed the thresholds. Permanent crossing would indicate that something about the variable is now different, or the variable itself is out of control. The dSP variable does cross the line 60-80% of its existence (somewhere around 1970 to around 2000), but it has come back to a stable area. The horizontal line at 0 is a good indication of stability.

```
plot(cusum_test_dearn, main = "dEARN")
```



This Cusum plot is much more stable than the previous. Earnings almost never spiral out of control (the only time it did was 2008 financial crisis), and most of the time the line is very close to 0. This would indicate that the Real Earnings are not only stable but consistent. Their tracking keeps them accurate. This is exactly how Real variables are supposed to behave.

Question 6

When comparing all of the models we looked at for this project, we see a substantial amount of variation between each one. Whether it be an AR() model, and ARDL() model, or the VAR() model, no one model is always best, and no one model is always worst. In order to choose a favorite model per response variable, we must look at each aspect of all models and compare the field.

Overall, I found that SP500 seemed more correlated to itself than EARN. Its forecasts were consistently more accurate, and the FEVD confirmed this hunch. I also found that Dividends played more of a factor in both SP500 and Earn than interest rates did, which went against my initial predictions. Interest affects the markets greatly, but did not seem to affect the month to month values and earnings of the stock market. There were many models made to predict each response variable, but each response variable had one best model.

Starting with SP500, the best model created for this response variable was the VAR model. This model had the most information to go through, but I found the combination of the IRF plot and the low RMSE very intriguing. The IRF provided insight into how much this variable changed as a result of the other response variable (EARN), and the low rmse ensured that the model stayed consistent throughout the entire range of the data. This model also had a low AIC score, which is necessary, and although the CUSUM plot wasn't ideal, the line does bend back towards 0 at the end, so all issues between 0.6 and 0.8 can be blamed on time period issues and not on the Variable itself. If I were using this dataset to predict future S&P500 values with money on the line, I would use a VAR model to ensure maximum accuracy.

For the EARN variable, VAR also proved to be most accurate. Much like the SP data, the IRF plot revealed a lot about the slow changing nature of Earnings. I especially liked the ACF and PACF spike at 12, which could be interpreted as large annual correlation despite weaker month-to-month correlation. The CUSUM plot in this case is much stronger than the SP CUSUM plot. The cumulative Earnings seem to stay constant, and the only time they fell outside of the red lines was during the 2008 Financial Crisis, which was a time where many financial indicators were unpredictable. The FEVD shows that although a large majority of EARN variance comes from itself, more information is needed to fully explain the variable, hence the VAR model.

Overall, this project made for a very unique modeling experience. Although not all of my code outputted in an expected manner, the modeling strength and corresponding figures taught me a lot about time-series modeling, which is something I had never attempted until now.